



KUWENI THE STORE

PROJECT DOCUMENTATION

A complete technical record of the desktop point-of-sale system, covering architecture, interfaces, database design, inventory, invoices, reporting and user management.

01

POINT OF SALE

Invoices & payments

02

INVENTORY

Products & stock

03

MANAGEMENT

Users & reports

Java Swing

MySQL

JasperReports

NetBeans

**SELL. TRACK. REPORT.**

Reliable store operations

PREPARED BY

Tevinu Manuditha Wijesinghe

Software Engineering Project

PROJECT

Kuweni The Store · Desktop POS

Sales · Inventory · Invoices · Reports

Table of Contents

1. Overall Description	3
1.1. Admin Capacities	3
1.2. Employee Capacities	3
2. ER Diagram	4
3. Interfaces	5
3.1. Sign In	7
3.2. Admin Dashboard	9
3.3. Employee Dashboard	11
3.4. Employee frame	13
3.5. Employee Dialog	18
3.6. Employee Update	24
3.7. Customer Registration	29
3.8. Customer Update Dialog	37
3.9. Product panel	42
3.10. Add product frame	46
3.11. Invoice view	53
3.12. Generate invoice	55
4. Loggers	61
5. Product Report	62
6. Invoice Report	63
7. USE CASE DIAGRAM	64

1. Overall Description

This document is meant for Mr.Charitha Rakshitha Attalage.This copyright of this document rests with Tevinu (without all logos, images.It's rests with Mr.Charitha.) and no part of the same should be copied without consulting with the same.This is the proposal document for "Kuweni The Store" service offering in the point of sale application design and development and custom POS application development space.

This Application give access only for admins and employees.

1.1. Admin Capacities

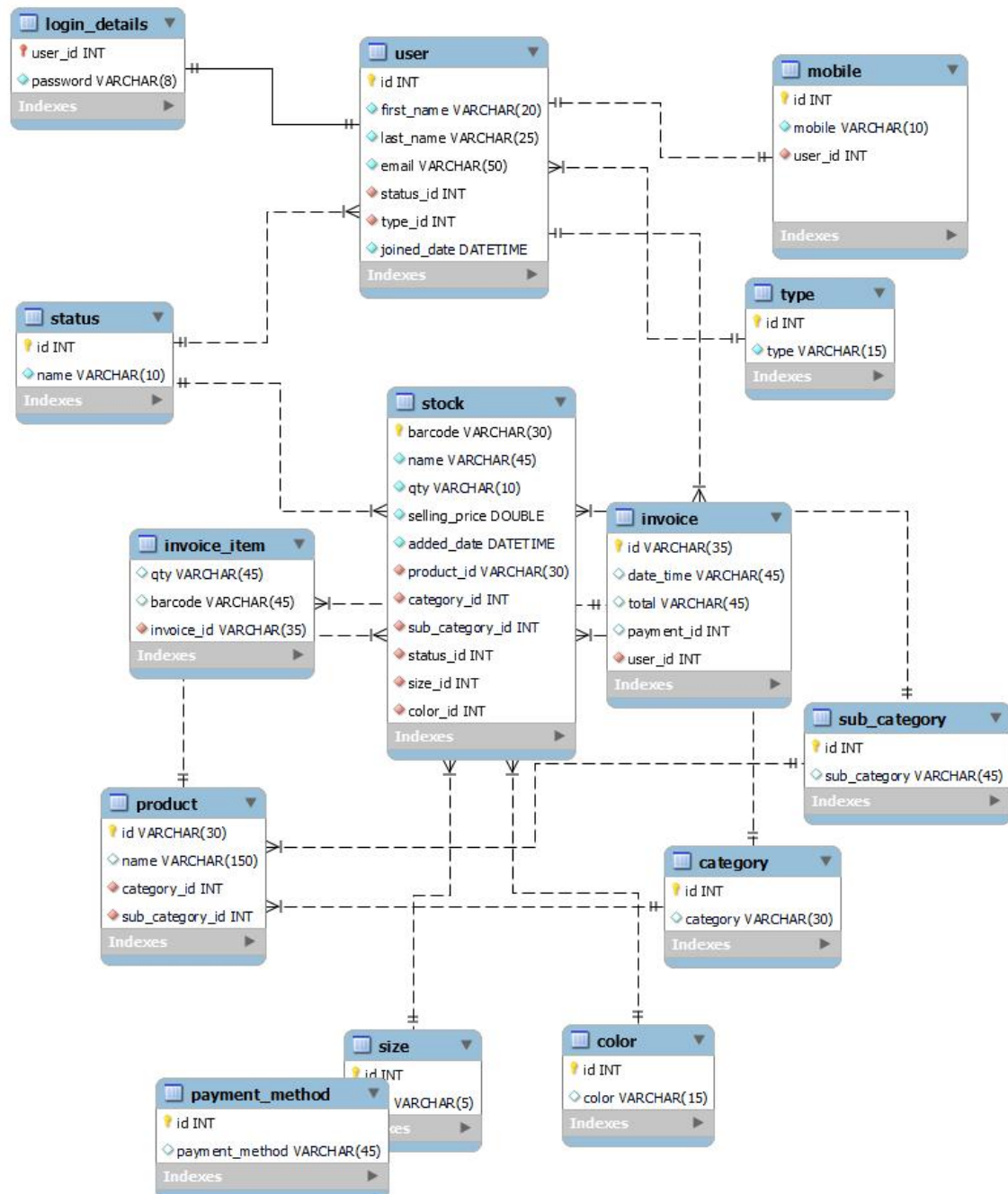
- Employee management,
 1. Employee registration.
 2. Employee update.
 3. Employee search.
 4. Employee delete.
- Customer management,
 1. Customer registration.
 2. Customer update.
 3. Customer search.
- Product management,
 1. Add product.
- Stock management,
 1. Add stock.
 2. Stock update.
 3. Stock delete.
 4. Stock search.
 5. Print stock report.
- Invoice management,
 1. Add invoice.
 2. Invoice search.
 3. Print invoice.
- Access dashboard.

1.2. Employee Capacities

- Customer management,
 1. Customer registration.
 2. Customer update.
 3. Customer search.
- Product management,
 1. Add product.
- Stock management,
 6. Add stock.
 7. Stock update.
 8. Stock delete.
 9. Stock search.
 10. Print stock report.

- Invoice management,
 4. Add invoice.
 5. Invoice search.
 6. Print invoice.
- Access dashboard.

2. ER Diagram



3. Interfaces

Splash



Description

This is the splash, which show when open the software. It has progress bar which fills until loading software files. When progress bar full fill directly opening the sign in frame and destroy splash.

Code

```
import com.formdev.flatlaf.extras.FlatSVGIcon;
import com.formdev.flatlaf.themes.FlatMacDarkLaf;
import java.io.IOException;
import java.text.SimpleDateFormat;
import java.util.Date;
import raven.toast.Notifications;
import java.util.logging.*;

public class Splash extends javax.swing.JFrame {
    public static Logger logger;
    private static Handler handler;

    public Splash() {
        initComponents();
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        progress();
        try {
            logger = Logger.getLogger(Splash.class.getName());
            handler = new FileHandler(String.valueOf(sdf.format(new Date()))+"log.1.log");
```

```

        //handler.setFormatter(new SimpleFormatter());
        logger.addHandler(handler);
    } catch (IOException ex) {
        ex.printStackTrace();
    }
    init();
    init1();
}
private void init(){
    logo.setIcon(new FlatSVGIcon("lk//teveram//kuweni//img//kuweni_2.svg",
    logo.getWidth(),logo.getHeight()));
    Notifications.getInstance().setJFrame(this);
}
private void init1(){
    pic.setIcon(new FlatSVGIcon("lk//teveram//kuweni//img//og.svg",
    pic.getWidth(),pic.getHeight()));
    Notifications.getInstance().setJFrame(this);
}
private void progress(){
    new Thread()->{
        for (int i = 0; i <= 100; i++){
            jProgressBar1.setValue(i);

            switch (i) {
                case 10:
                    jLabel1.setText("Loading...");
                    break;
                case 37:
                    jLabel1.setText("Database connection create...");
                    break;
                case 50:
                    jLabel1.setText("Reports Libraries Loading...");
                    break;
                case 80:
                    jLabel1.setText("Application Loading Success");
                    break;
                case 95:
                    jLabel1.setText("Done");
                    break;
            }
        }
        try {
            Thread.sleep(30);
        } catch (InterruptedException e) {
            Splash.logger.warning(e.getMessage());
        }
    }
    new signIn().setVisible(true);
    this.dispose();
}

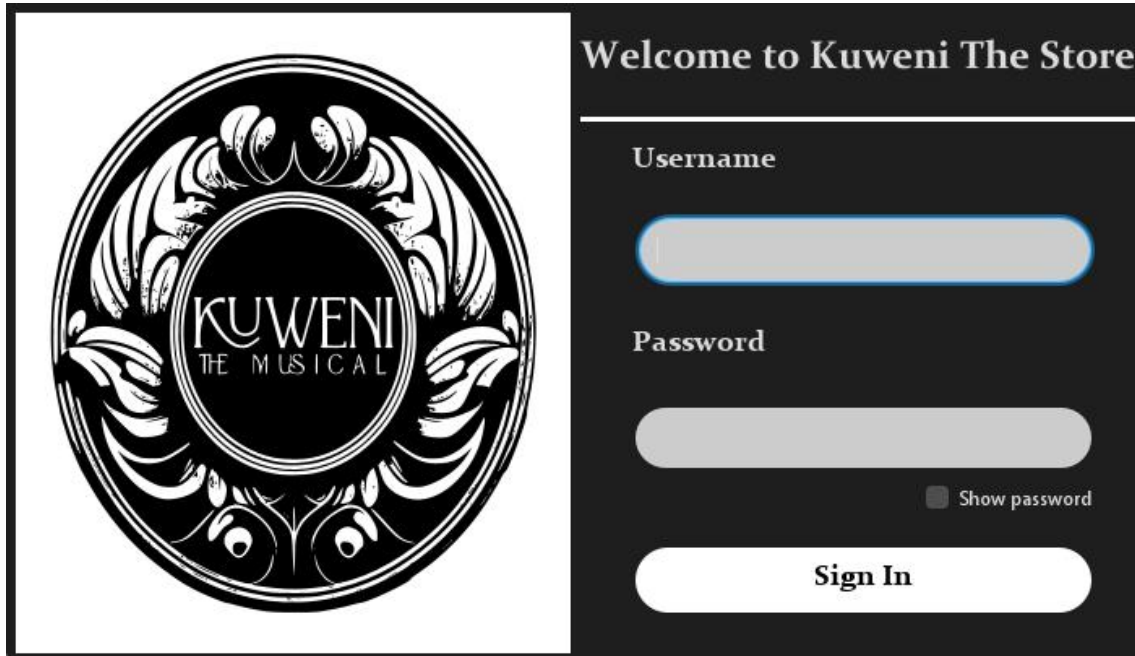
```

```

    }).start();
}

```

3.1. Sign In



Description

The sign In frame is the real pass to the application, because this frame requires admin or employee id as username and password. These two fields are validated, then user must have to provide valid username and password. Also user can see or hide the characters of password field by using show password check box. When user clicks the sign in button after providing, get the user type on database and if the type is employee, directly employee dashboard. If type is admin, moves to admin dashboard.

Code

```

import com.formdev.flatlaf.extras.FlatSVGIcon;
import com.formdev.flatlaf.themes.FlatMacDarkLaf;
import java.awt.Color;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JOptionPane;
import lk.teveram.kuweni.connection.MySQL;
import raven.toast.Notifications;
public class signIn extends javax.swing.JFrame {
    public signIn() {
        initComponents();
        init();
    }
    private void init(){
        this.setBackground(new Color(0,0,0,0));
        loginImage.setIcon(new FlatSVGIcon("lk//teveram//kuweni//img//kuweni_1.svg",
            loginImage.getWidth(),loginImage.getHeight()));
        Notifications.getInstance().setJFrame(this);
    }
}

```

```

private void roundButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    String id = roundTextField1.getText();
    String pw = String.valueOf(roundPasswordField2.getPassword());
    if(id.isEmpty()){
        Notifications.getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Username is required");
    }else if(pw.isEmpty()){
        Notifications.getInstance().
            show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Password is required");
    }else{
        try {
            ResultSet login = MySQL.search("SELECT * FROM `login_details` "
                + "WHERE `user_id`='"+id+"' AND `password`='"+pw+"'");
            if(login.next()){
                ResultSet details = MySQL.search("SELECT * FROM `user` "
                    + "INNER JOIN `status` ON `user`.`status_id`"
                    + "INNER JOIN `type` ON `user`.`type_id`"
                    + "WHERE `user`.`id`='"+login.getString("user_id")+"'");
                if(details.next()){
                    String type = details.getString("type");
                    if(type.equals("ADMIN")){
                        AdminDashboard adminDashboard = new AdminDashboard();
                        adminDashboard.setVisible(true);
                        this.dispose();
                    }else{
                        EmployeeDashboard employeeDashboard = new EmployeeDashboard();
                        employeeDashboard.setVisible(true);
                        this.dispose();
                    }
                }
            }else{
                JOptionPane.showMessageDialog(this,"Provide valid details!", id, HEIGHT);
            }
        } catch (SQLException e) {
            Splash.logger.warning(e.getMessage());
        }
    }
}

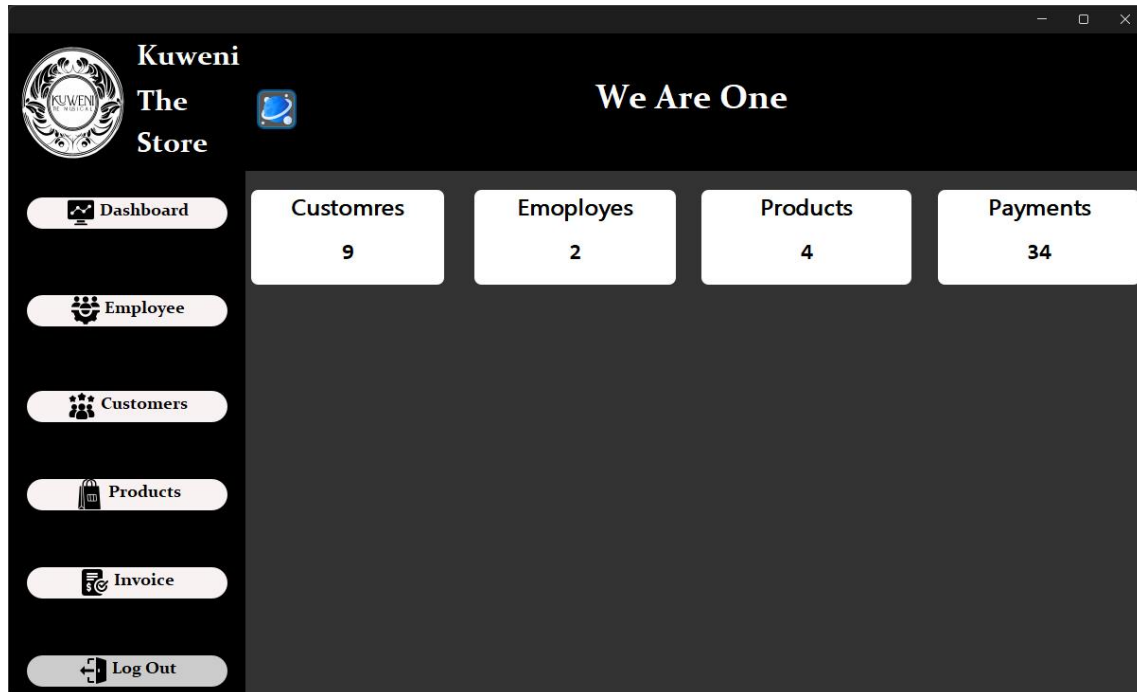
private void jCheckBox1ActionPerformed(java.awt.event.ActionEvent evt) {
    if (jCheckBox1.isSelected()) {
        roundPasswordField2.setEchoChar('\u0000');
    } else {
        roundPasswordField2.setEchoChar('\u2022');
    }
}

```



```
}
}
```

3.2. Admin Dashboard



Description

The admin dashboard has buttons for customer management (customer), employee management (Employee), and there is a toggle button for hide that buttons. On the top of the dashboard, it shows "We Are One", which is a main artistic concept of all people. And when you click the logout button, it closes the admin dashboard and opens the sign-in frame. The default admin dashboard shows the dashboard panel. It contains the count of customers, employees, products, and payments of this application.

Code

```
import com.formdev.flatlaf.FlatLaf;
import com.formdev.flatlaf.extras.FlatSVGIcon;
import com.formdev.flatlaf.themes.FlatMacDarkLaf;
import java.awt.BorderLayout;
import java.awt.Dimension;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.SwingUtilities;
import lk.teveram.kuweni.panel.CustomerRegistration;
import lk.teveram.kuweni.panel.Employee;
import lk.teveram.kuweni.panel.Dashboard;
import lk.teveram.kuweni.panel.Invoices;
import lk.teveram.kuweni.panel.Products;
import raven.toast.Notifications;
```

```
public class AdminDashboard extends JFrame {
    public AdminDashboard() {
```

```

        initComponents();
        init();
        loadPanel(new Dashboard());
    }
    private void init(){
        logo.setIcon(new FlatSVGIcon("lk//teveram//kuweni//img//kuweni_2.svg",
        logo.getWidth(),logo.getHeight()));
        Notifications.getInstance().setJFrame(this);
    }
    private void loadPanel(JPanel panel){
        containLoader.removeAll();
        if(!panel.isShowing()){
            containLoader.add(panel, BorderLayout.CENTER);
            FlatLaf.updateUI();
        }
    }
    private void jToggleButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        if(jToggleButton1.isSelected()){
            new Thread()->{
                for (int i= jPanel1.getWidth(); i>=50; i--){
                    jPanel1.setPreferredSize(new Dimension(i,jPanel1.getHeight()));
                    SwingUtilities.updateComponentTreeUI(jPanel1);
                    try {
                        Thread.sleep(3);
                    } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                    }
                }
            }.start();
        }else {new Thread()->{
            for (int i= jPanel1.getWidth(); i<=250; i++){
                jPanel1.setPreferredSize(new Dimension(i,jPanel1.getHeight()));
                SwingUtilities.updateComponentTreeUI(jPanel1);
                try {
                    Thread.sleep(3);
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
        }.start();
    }
}
private void formWindowOpened(java.awt.event.WindowEvent evt) {
    this.setExtendedState(JFrame.MAXIMIZED_BOTH);
}
private void roundButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    loadPanel(new Dashboard());
}

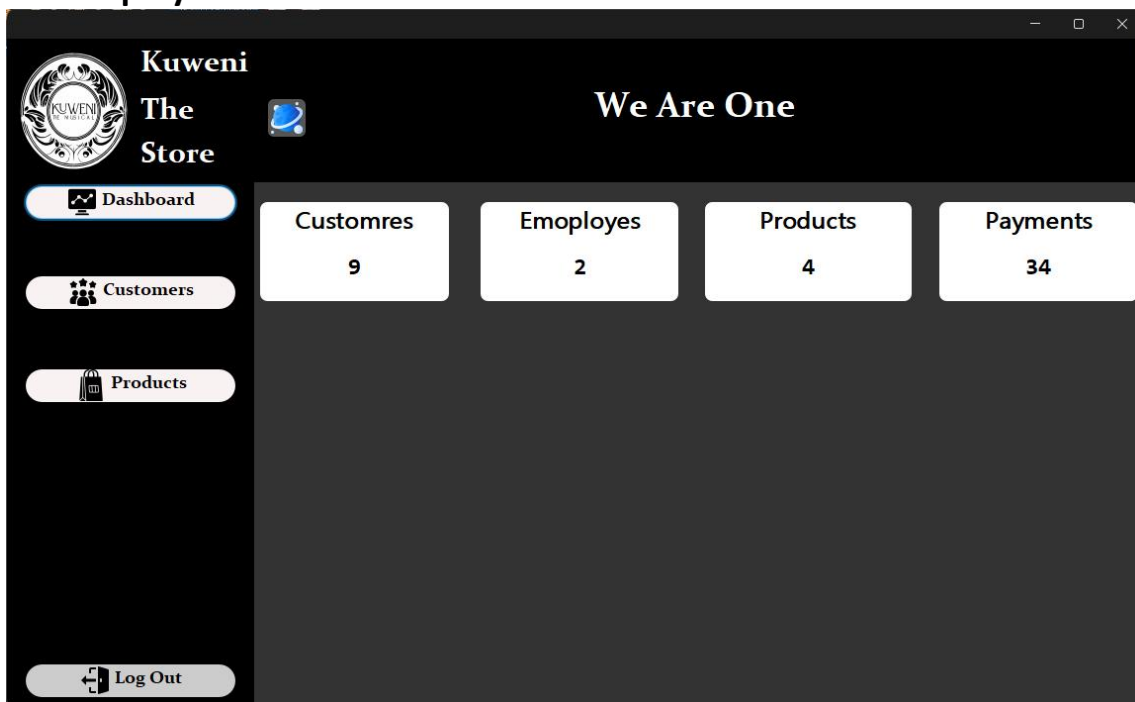
```

```

private void roundButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    loadPanel(new Employee(this));
}
private void roundButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    loadPanel(new CustomerRegistration(this));
}
private void roundButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    loadPanel(new Products(this));
}
private void roundButton5ActionPerformed(java.awt.event.ActionEvent evt) {
    loadPanel(new Invoices());
}
private void roundButton6ActionPerformed(java.awt.event.ActionEvent evt) {
    signIn SignIN = new signIn();
    SignIN.setVisible(true);
    this.dispose();
}

```

3.3. Employee Dashboard



Code

```

import com.formdev.flatlaf.FlatLaf;
import com.formdev.flatlaf.extras.FlatSVGIcon;
import com.formdev.flatlaf.themes.FlatMacDarkLaf;
import java.awt.BorderLayout;
import java.awt.Dimension;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.SwingUtilities;
import lk.teveram.kuweni.panel.CustomerRegistration;
import lk.teveram.kuweni.panel.Dashboard;

```

```

import lk.teveram.kuweni.panel.Products;
import raven.toast.Notifications;

public class EmployeeDashboard extends javax.swing.JFrame {
    public EmployeeDashboard() {
        initComponents();
        init();
        loadPanel(new Dashboard());
    }
    private void init(){
        //this.setBackground(new Color(0,0,0,0));
        logo.setIcon(new FlatSVGIcon("lk//teveram//kuweni//img//kuweni_2.svg",
        logo.getWidth(),logo.getHeight()));
        Notifications.getInstance().setJFrame(this);
    }
    private void loadPanel(JPanel panel){
        containLoader.removeAll();
        if(!panel.isShowing()){
            containLoader.add(panel, BorderLayout.CENTER);
            FlatLaf.updateUI();
        }
    }
    private void roundButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        loadPanel(new Dashboard());
    }
    private void roundButton3ActionPerformed(java.awt.event.ActionEvent evt) {
        loadPanel(new CustomerRegistration(this));
    }
    private void roundButton4ActionPerformed(java.awt.event.ActionEvent evt) {
        Products products = new Products(this);
        products.newButton.setEnabled(false);
        loadPanel(products);
    }
    private void jToggleButton1ActionPerformed(java.awt.event.ActionEvent evt) {
        if(jToggleButton1.isSelected()){
            new Thread()->{
                for (int i= jPanel1.getWidth(); i>=50; i--){
                    jPanel1.setPreferredSize(new Dimension(i,jPanel1.getHeight()));
                    SwingUtilities.updateComponentTreeUI(jPanel1);
                    try {
                        Thread.sleep(3);
                    } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                    }
                }
            }.start();
        }else {new Thread()->{
            for (int i= jPanel1.getWidth(); i<=250; i++){

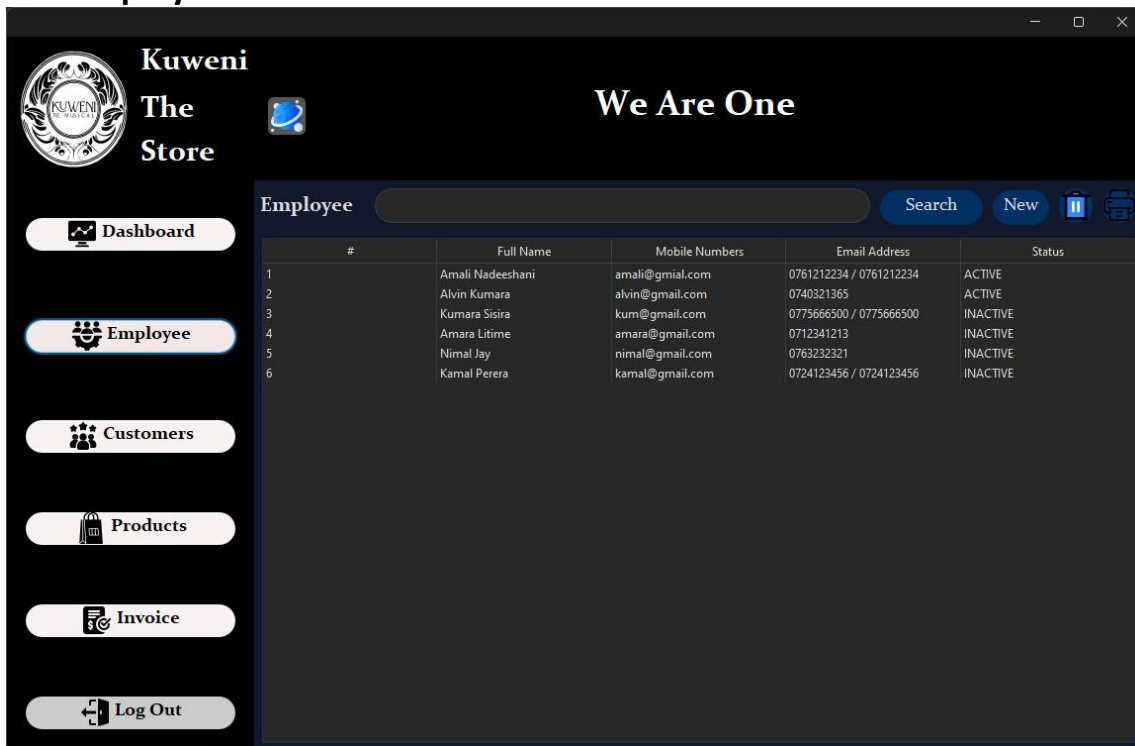
```

```

jPanel1.setPreferredSize(new Dimension(i,jPanel1.getHeight()));
SwingUtilities.updateComponentTreeUI(jPanel1);
try {
    Thread.sleep(3);
} catch (InterruptedException e) {
    throw new RuntimeException(e);
}
}
}).start();
}
}
private void formWindowOpened(java.awt.event.WindowEvent evt) {
    this.setExtendedState(JFrame.MAXIMIZED_BOTH);
}
}

```

3.4. Employee frame



Description

The employee panel shows all the details of employees. User can add new employee by clicking new button. And can delete employee by pressing delete button. User can update employee details by clicking a row two times. When user typing in the search bar, it's searching the employees and update table.

Code

```

import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Vector;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;

```

```

import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.dialog.EmployeeDialog;
import lk.teveram.kuweni.gui.Splash;

public class Employee extends javax.swing.JPanel {
    private final java.awt.Frame Dashboard;
    public Employee(java.awt.Frame dashboard) {
        initComponents();
        this.Dashboard = dashboard;
        loadTable();
    }
    public void loadTable(){
        try {
            ResultSet result = MySQL.search("SELECT* FROM `user` "
                + "INNER JOIN `status` ON `user`.`status_id`=`status`.`id`"
                + "WHERE `user`.`type_id`='2'");
            DefaultTableModel tableModel = (DefaultTableModel)
employeeTable.getModel();
            tableModel.setRowCount(0);
            int count =0;
            while(result.next()){
                count++;
                Vector<String> employeeList = new Vector();
                employeeList.add(String.valueOf(count));
                employeeList.add(result.getString("first_name")+
"+result.getString("last_name"));
                employeeList.add(result.getString("email"));
                ResultSet contacts = MySQL.search("SELECT * FROM `mobile` "
                    + "WHERE `user_id`='"+result.getString("id")+"'");
                String mobile="";
                boolean bool = true;
                while(contacts.next()){
                    if(bool){
                        mobile=contacts.getString("mobile");
                        bool = false;
                    }else{
                        mobile+=" / "+contacts.getString("mobile");
                    }
                }
                employeeList.add(mobile);
                employeeList.add(result.getString("name"));
                tableModel.addRow(employeeList);
            }
        } catch (SQLException e) {
            Splash.logger.warning(e.getMessage());
        }
    }
    private void employeeNewActionPerformed(java.awt.event.ActionEvent evt) {

```

```

EmployeeDialog employeeDialog = new EmployeeDialog(Dashboard,true);
employeeDialog.setLocationRelativeTo(Dashboard);
employeeDialog.updateEmployee.setEnabled(false);
employeeDialog.statusCombo.setEnabled(false);
employeeDialog.setEmployeePanel(this);
    employeeDialog.setVisible(true);
}
private void employeeTableMouseClicked(java.awt.event.MouseEvent evt) {
    if(evt.getClickCount()==2){
        int selectedRow = employeeTable.getSelectedRow();
        String fullName= (String)(employeeTable.getValueAt(selectedRow, 1));
        String email = (String)employeeTable.getValueAt(selectedRow,2);
        String mobile = (String)employeeTable.getValueAt(selectedRow,3);
        String status = (String) (employeeTable.getValueAt(selectedRow, 4));
        String type="";
        try {
            ResultSet result = MySQL.search("SELECT `type` FROM `type` "
                + "WHERE `id`=(SELECT `type_id` FROM `user` "
                + "WHERE `email`='"+email+"')");
            if(result.next()){
                type = result.getString("type");
                System.out.println(type);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        String[] split = fullName.split(" ");
        String firstName = split[0];
        String lastName = split[1];
        String mobileOne = "";
        String mobileTwo = "";
        if(mobile.contains("/")){
            String[] Mobile = mobile.split(" / ");
            mobileOne = Mobile[0];
            mobileTwo = Mobile[1];
        }else{
            mobileOne = mobile;
        }
        EmployeeDialog employeeDialog = new EmployeeDialog(Dashboard, true);
        employeeDialog.setLocationRelativeTo(Dashboard);
        employeeDialog.getLastNameInput().setText(lastName);
        employeeDialog.getEmailInput().setText(email);
        employeeDialog.getMobileOneInput().setText(mobileOne);
        employeeDialog.getMobileTwoInput().setText(mobileTwo);
        employeeDialog.getStatusCombo().setSelectedItem(status);
        employeeDialog.addEmployee.setEnabled(false);
        employeeDialog.setEmployeePanel(this);
        employeeDialog.setVisible(true);
    }
}

```

```

    }
}

private void roundButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    String text = searchInput.getText();
    if(!text.isBlank()){
        try{
            ResultSet result = MySQL.search("SELECT * FROM `user` INNER JOIN `type` "
                + "ON `user`.`type_id`=`type`.`id` INNER JOIN `status` "
                + "ON `user`.`status_id`=`status`.`id`"
                + "WHERE (`first_name` LIKE '"+text+"%' "
                + "OR `email` LIKE '"+text+"%') AND `type`='CASHIER'");
            DefaultTableModel                                tableModel                                =
            (DefaultTableModel)employeeTable.getModel();
            tableModel.setRowCount(0);
            int count =0;
            while(result.next()){
                Vector<String> search = new Vector();
                count++;
                search.add(String.valueOf(count));
                search.add(result.getString("first_name")+" "+result.getString("last_name"));
                search.add(result.getString("email"));
                ResultSet mobileResult = MySQL.search("SELECT * FROM `mobile` "
                    + "WHERE `user_id`='"+result.getString("id")+"'");
                boolean bool = true;
                String mobile = "";
                while(mobileResult.next()){
                    if(bool){
                        mobile = mobileResult.getString("mobile");
                        bool = false;
                    }else{
                        mobile+=" / "+mobileResult.getString("mobile");
                    }
                }
                search.add(mobile);
                search.add(result.getString("name"));
                tableModel.addRow(search);
            }
        }catch (SQLException e){
            Splash.logger.warning(e.getMessage());
        }
        }else{
            loadTable();
        }
    }

private void searchInputKeyReleased(java.awt.event.KeyEvent evt) {
    String text = searchInput.getText();

```



```

if(!text.isBlank()){
try{
    ResultSet result = MySQL.search("SELECT * FROM `user` INNER JOIN `type` "
        + "ON `user`.`type_id`=`type`.`id` INNER JOIN `status` "
        + "ON `user`.`status_id`=`status`.`id`"
        + "WHERE (`first_name` LIKE '"+text+"%' "
        + "OR `email` LIKE '"+text+"%') AND `type`='CASHIER'");
    DefaultTableModel                                tableModel                                =
(DefaultTableModel)employeeTable.getModel();
    tableModel.setRowCount(0);
    int count =0;
    while(result.next()){
        Vector<String> search = new Vector();
        count++;
        search.add(String.valueOf(count));
        search.add(result.getString("first_name")+" "+result.getString("last_name"));
        search.add(result.getString("email"));
        ResultSet mobileResult = MySQL.search("SELECT * FROM `mobile` "
            + "WHERE `user_id`='"+result.getString("id")+"'");
        boolean bool = true;
        String mobile = "";
        while(mobileResult.next()){
            if(bool){
                mobile = mobileResult.getString("mobile");
                bool = false;
            }else{
                mobile+=" / "+mobileResult.getString("mobile");
            }
        }
        search.add(mobile);
        search.add(result.getString("name"));
        tableModel.addRow(search);
    }

}catch (SQLException e){
    Splash.logger.warning(e.getMessage());
}
}else{
    loadTable();
}

}

private void employeeNew2ActionPerformed(java.awt.event.ActionEvent evt) {
    if (employeeTable.getSelectedRowCount() == 1) {
        int response = JOptionPane.showConfirmDialog(
            this, "Are you sure you want to delete this employee?",
            "Warning", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE
        );
    }
}

```

```

if (response == JOptionPane.YES_OPTION) {
    int selectedRow = employeeTable.getSelectedRow();
    String userEmail = (String) employeeTable.getValueAt(selectedRow, 2);

    MySQL.iud("DELETE `mobile` FROM `mobile` " +
        "INNER JOIN `user` ON `mobile`.`user_id` = `user`.`id` " +
        "INNER JOIN `type` ON `user`.`type_id` = `type`.`id` " +
        "WHERE `user`.`email` = '" + userEmail + "' " +
        "AND `type`.`type` = 'CASHIER'");

    MySQL.iud("DELETE `user` FROM `user` " +
        "INNER JOIN `type` ON `user`.`type_id` = `type`.`id` " +
        "WHERE `user`.`email` = '" + userEmail + "' " +
        "AND `type`.`type` = 'CASHIER'");
    loadTable();
    searchInput.setText("");
    JOptionPane.showMessageDialog(this, "Employee deleted successfully",
        "Information", JOptionPane.INFORMATION_MESSAGE);
}
}
}

```

3.5. Employee Dialog

Code

```

import com.formdev.flatlaf.FlatDarkLaf;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Vector;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JComboBox;
import javax.swing.JOptionPane;
import lk.teveram.kuweni.component.RoundTextField;
import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.gui.Splash;
import lk.teveram.kuweni.panel.Employee;
import lk.teveram.kuweni.validation.Validation;
import raven.toast.Notifications;
public class EmployeeDialog extends javax.swing.JDialog {
    private Employee employee;

    public EmployeeDialog(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        updateUserStatusCombo();
    }
    public void setEmployeePanel(Employee ep){
        this.employee=ep;
    }
    private void addEmployeeActionPerformed(java.awt.event.ActionEvent evt) {
        SimpleDateFormat dateFormatter = new SimpleDateFormat
        ("yyyy-MM-dd HH:mm:ss");
        String fname = firstNameInput.getText();
        String lname = lastNameInput.getText();
        String mobileOne = mobileOneInput.getText();
        String mobileTwo = mobileTwoInput.getText();
        String email = emailInput.getText();
        int status =(int) statusCombo.getSelectedIndex();
        if(nameValidation(fname)){
            return;
        }
        if(nameValidation(lname)){
            return;
        }
        if(emailValidation(email)){
            return;
        }
        if(mobileOne.isEmpty()){
            Notifications.getInstance().show
            (Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT,

```

```

        "Mobile number is required!");
    }else if(mobileValidation(mobileOne)){
        return;
    }
    if(!mobileTwo.isEmpty()){
        if(mobileValidation(mobileTwo)){
            return;}
    }
    if(status <= 0){
        Notifications.getInstance().show
        (Notifications.Type.WARNING,
        Notifications.Location.TOP_RIGHT,
        "Select status!");
    }
    try {
        ResultSet resultSet = MySQL.search("SELECT * FROM `user` "
            + "WHERE `email`='"+email+"'");
        if(resultSet.next()){
            JOptionPane.showMessageDialog
            (this, "Customer already registered", "Warning",
            JOptionPane.WARNING_MESSAGE
        }else{
            String joinedDate = dateFormatter.format(new Date());
            MySQL.iud("INSERT INTO `user` (`first_name`, `last_name`, `email`, "
                + "`status_id`, `type_id`, `joined_date`) " +
                "VALUES ('" + fname + "', '" + lname + "', '" + email + "', "
                + "'1', '2', '" + joinedDate + "')");
            ResultSet contactDetails = MySQL.search("SELECT `id` FROM `user`"
                + " WHERE `email`='"+email+"'");
            if(contactDetails.next()){
                String userId = contactDetails.getString("id");
                MySQL.iud("INSERT INTO `mobile` (`mobile`, `user_id`) "
                    + "VALUES ('"+mobileOne+"', '"+userId+"')");
            if(!mobileTwo.isEmpty()){
                MySQL.iud("INSERT INTO `mobile` (`mobile`, `user_id`) "
                    + "VALUES ('"+mobileTwo+"', '"+userId+"')");
            }
            JOptionPane.showMessageDialog(this,
                "Employee registration successfull", "Success",
                JOptionPane.INFORMATION_MESSAGE);
            this.dispose();
            employee.loadTable();
        }
    }
} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}

```

```

}
private void updateUserStatusCombo() {
    try {
        ResultSet result = MySQL.search("SELECT * FROM `status`");
        Vector<String> statusList = new Vector();
        statusList.add("Select status");
        while (result.next()) {
            statusList.add(result.getString("name"));
        }
        DefaultComboBoxModel comboBoxMode = new
DefaultComboBoxModel(statusList);
        this.statusCombo.setModel(comboBoxMode);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void updateEmployeeActionPerformed(java.awt.event.ActionEvent evt) {
    String firstName = firstNameInput.getText();
    String lastName = lastNameInput.getText();
    String email = emailInput.getText();
    String mobileOne = mobileOneInput.getText();
    String mobileTwo = mobileTwoInput.getText();
    int status = statusCombo.getSelectedIndex();
    if(nameValidation(firstName)){
        return;
    }
    if(nameValidation(lastName)){
        return;
    }
    if(emailValidation(email)){
        return;
    }
    if (mobileOne.isEmpty()) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Mobile one is required!");
    } else {
        if (mobileValidation(mobileOne)) {
            return;
        }
    }
    if (!mobileTwo.isEmpty()) {
        if (mobileValidation(mobileTwo)) {
            return;
        }
    }
}

```

```

        MySQL.iud("UPDATE `user` INNER JOIN `mobile` SET
`user`.`first_name`='"+firstName+"",
        + "`user`.`last_name`='"+lastName+"", `user`.`type_id`='2',"
        + "`user`.`status_id`='"+status+"WHERE `user`.`email`='"+email+"");
    try {
        ResultSet currentUser = MySQL.search("SELECT `id` FROM `user` WHERE
`email`='"+ email + "'");
        String userId="";
        if (currentUser.next()){
            userId = currentUser.getString("id");
        }
        ResultSet contacts = MySQL.search("SELECT * FROM mobile "
            + "WHERE user_id='"+userId+"");
        byte increment = 0;
        while(contacts.next()){
            MySQL.iud("UPDATE `mobile` SET `mobile`='"+mobileOne+" "
                + "WHERE `user_id`='"+userId+"");
            if(increment > 0){
                MySQL.iud("UPDATE `mobile` SET `mobile`='"+mobileTwo+"""
                    + " WHERE `user_id`='"+userId+"");
            }
            increment++;
        }
        JOptionPane.showMessageDialog(this, "Update successful!", "Warning",
            JOptionPane.WARNING_MESSAGE);
        this.dispose();
        employee.loadTable();
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private boolean nameValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "This field is required!");
        return true;
    }
    return false;
}

private boolean emailValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Email is required!");
        return true;
    }else if(!value.matches(Validation.EMAIL_VALIDATION.validate())){
        Notifications.getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,

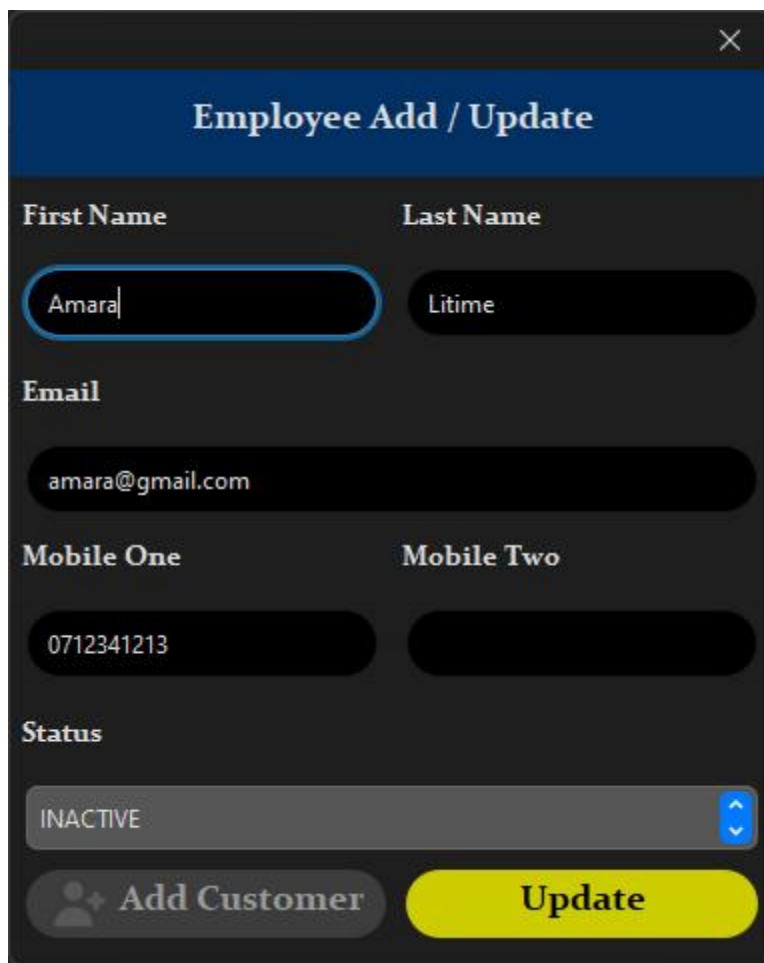
```

```

        "Please enter valid email");{
    return true;
}
}
    return false;
}
private boolean mobileValidation(String value) {
    if (!value.matches(Validation.MOBILE_VALIDATION.validate())) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Provide valid mobile number");
        return true;
    }
    return false;
}
public RoundTextField getFirstNameInput() {
    return firstNameInput;
}
public RoundTextField getLastNameInput() {
    return lastNameInput;
}
public RoundTextField getEmailInput() {
    return emailInput;
}
public RoundTextField getMobileOneInput() {
    return mobileOneInput;
}
public RoundTextField getMobileTwoInput() {
    return mobileTwoInput;
}
public JComboBox<String> getStatusCombo() {
    return statusCombo;
}
}

```

3.6. Employee Update



The image shows a Java Swing dialog box titled "Employee Add / Update". It has a dark theme with a blue header bar. The dialog contains several input fields: "First Name" (containing "Amara"), "Last Name" (containing "Litime"), "Email" (containing "amara@gmail.com"), "Mobile One" (containing "0712341213"), and "Mobile Two" (empty). There is a "Status" dropdown menu currently set to "INACTIVE". At the bottom, there are two buttons: "Add Customer" (disabled, with a plus icon) and "Update" (active, in yellow).

Code

```
import com.formdev.flatlaf.FlatDarkLaf;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Vector;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JComboBox;
import javax.swing.JOptionPane;
import lk.teveram.kuweni.component.RoundTextField;
import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.gui.Splash;
import lk.teveram.kuweni.panel.Employee;
import lk.teveram.kuweni.validation.Validation;
import raven.toast.Notifications;
public class EmployeeDialog extends javax.swing.JDialog {
    private Employee employee;
    public EmployeeDialog(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        updateUserStatusCombo();
    }
}
```



```

}
public void setEmployeePanel(Employee ep){
    this.employee=ep;
}
private void addEmployeeActionPerformed(java.awt.event.ActionEvent evt) {
    SimpleDateFormat dateFormatter = new SimpleDateFormat
    ("yyyy-MM-dd HH:mm:ss");
    String fname = firstNameInput.getText();
    String lname = lastNameInput.getText();
    String mobileOne = mobileOneInput.getText();
    String mobileTwo = mobileTwoInput.getText();
    String email = emailInput.getText();
    int status =(int) statusCombo.getSelectedIndex();
    if(nameValidation(fname)){
        return;
    }
    if(nameValidation(lname)){
        return;
    }
    if(emailValidation(email)){
        return;
    }
    if(mobileOne.isEmpty()){
        Notifications.getInstance().show
        (Notifications.Type.WARNING,
        Notifications.Location.TOP_RIGHT,
        "Mobile number is required!");
    }else if(mobileValidation(mobileOne)){
        return;
    }
    if(!mobileTwo.isEmpty()){
        if(mobileValidation(mobileTwo)){
            return;}
    }
    if(status <= 0){
        Notifications.getInstance().show
        (Notifications.Type.WARNING,
        Notifications.Location.TOP_RIGHT,
        "Select status!");
    }
    try {
        ResultSet resultSet = MySQL.search("SELECT * FROM `user` "
        + "WHERE `email`='"+email+"'");
        if(resultSet.next()){
            JOptionPane.showMessageDialog
            (this, "Customer already registered", "Warning",
            JOptionPane.WARNING_MESSAGE);
        }else{

```

```

String joinedDate = dateFormatter.format(new Date());
MySQL.iud("INSERT INTO `user` (`first_name`, `last_name`, `email`,
    + " `status_id`, `type_id`, `joined_date`) " +
    "VALUES ('" + fname + "', '" + lname + "', '" + email + "',"
    + " '1', '2', '" + joinedDate + "')");

ResultSet contactDetails = MySQL.search("SELECT `id` FROM `user`"
    + " WHERE `email`='" + email + "'");
if(contactDetails.next()){
    String userId = contactDetails.getString("id");
    MySQL.iud("INSERT INTO `mobile` (`mobile`, `user_id`) "
        + "VALUES ('" + mobileOne + "', '" + userId + "')");
    if(!mobileTwo.isEmpty()){
        MySQL.iud("INSERT INTO `mobile` (`mobile`, `user_id`) "
            + "VALUES ('" + mobileTwo + "', '" + userId + "')");
    }
    JOptionPane.showMessageDialog(this,
        "Employee registration successfull", "Success",
        JOptionPane.INFORMATION_MESSAGE);
    this.dispose();
    employee.loadTable();
}
} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}
}

private void updateUserStatusCombo() {
    try {
        ResultSet result = MySQL.search("SELECT * FROM `status`");
        Vector<String> statusList = new Vector();
        statusList.add("Select status");
        while (result.next()) {
            statusList.add(result.getString("name"));
        }
        DefaultComboBoxModel comboBoxMode = new
DefaultComboBoxModel(statusList);
        this.statusCombo.setModel(comboBoxMode);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void updateEmployeeActionPerformed(java.awt.event.ActionEvent evt) {
    String firstName = firstNameInput.getText();
    String lastName = lastNameInput.getText();
    String email = emailInput.getText();
    String mobileOne = mobileOneInput.getText();
    String mobileTwo = mobileTwoInput.getText();

```

```

int status = statusCombo.getSelectedIndex();
if(nameValidation(firstName)){
    return;
}
if(nameValidation(lastName)){
    return;
}
if(emailValidation(email)){
    return;
}
if (mobileOne.isEmpty()) {
    Notifications
        .getInstance()
        .show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT,
            "Mobile one is required!");
} else {
    if (mobileValidation(mobileOne)) {
        return;
    }
}
if (!mobileTwo.isEmpty()) {
    if (mobileValidation(mobileTwo)) {
        return;
    }
}
MySQL.iud("UPDATE `user` INNER JOIN `mobile` SET
`user`.`first_name`='"+firstName+"',
+ "`user`.`last_name`='"+lastName+"`, `user`.`type_id`='2',"
+ "`user`.`status_id`='"+status+"'WHERE `user`.`email`='"+email+"'");
try {
    ResultSet currentUser = MySQL.search("SELECT `id` FROM `user` WHERE
`email`='"+ email + "'");
    String userId="";
    if (currentUser.next()){
        userId = currentUser.getString("id");
    }
    ResultSet contacts = MySQL.search("SELECT * FROM mobile "
        + "WHERE user_id='"+userId+"'");
    byte increment = 0;
    while(contacts.next()){
        MySQL.iud("UPDATE `mobile` SET `mobile`='"+mobileOne+" "
            + "WHERE `user_id`='"+userId+"'");
        if(increment > 0){
            MySQL.iud("UPDATE `mobile` SET `mobile`='"+mobileTwo+" "
                + "WHERE `user_id`='"+userId+"'");
        }
        increment++;
    }
}

```

```

    }
    JOptionPane.showMessageDialog(this, "Update successful!", "Warning",
        JOptionPane.WARNING_MESSAGE);
    this.dispose();
    employee.loadTable();
} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}
}

private boolean nameValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "This field is required!");
        return true;
    }
    return false;
}

private boolean emailValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Email is required!");
        return true;
    }else if(!value.matches(Validation.EMAIL_VALIDATION.validate())){
        Notifications.getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Please enter valid email");
        return true;
    }
}

return false;
}

private boolean mobileValidation(String value) {
    if (!value.matches(Validation.MOBILE_VALIDATION.validate())) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Provide valid mobile number");
        return true;
    }
    return false;
}

public RoundTextField getFirstNameInput() {
    return firstNameInput;
}

public RoundTextField getLastNameInput() {
    return lastNameInput;
}

```

```

}
public RoundTextField getEmailInput() {
    return emailInput;
}
public RoundTextField getMobileOneInput() {
    return mobileOneInput;
}
public RoundTextField getMobileTwoInput() {
    return mobileTwoInput;
}
public JComboBox<String> getStatusCombo() {
    return statusCombo;
}
}

```

3.7. Customer Registration

#	Full Name	Email Address	Mobile Numbers	Status
1	Anuradhi Fernando	anuradhi@gmail.c...	0763026654	ACTIVE
2	Robert Nox	nox@gmail.com	0771698322	ACTIVE
3	Albert Jack	a@gmail.com	0773669874	ACTIVE
4	Trevor Gamage	trevor@gmail.com		ACTIVE
5	James Jack	james@gmail.com	0711245896 / 0711...	ACTIVE
6	Justin Perera	justin@gmail.com		ACTIVE
7	Chester Benington	chester@gmail.com		ACTIVE
8	Randy Gayan	randy@gmail.com	0771114425	INACTIVE
9	Ravi Silva	ravi@gmail.com	0765244200 / 0765...	INACTIVE

Description

In this frame, user can register customer, update customer and search customer. When user types a mobile number in the mobile one field, search a customer by the number on key released.

Code

```

import java.util.Vector;
import javax.swing.table.DefaultTableModel;
import lk.teveram.kuweni.connection.MySQL;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
import java.util.Date;
import javax.swing.JOptionPane;
import lk.teveram.kuweni.dialog.CustomerDialog;

```

```

import lk.teveram.kuweni.gui.Splash;
import lk.teveram.kuweni.gui.invoice;
import lk.teveram.kuweni.model.customerBean;
import lk.teveram.kuweni.validation.Validation;
import raven.toast.Notifications;
public class CustomerRegistration extends javax.swing.JPanel {
    private final java.awt.Frame Dashboard;
    private final invoice Invoice;
    public CustomerRegistration(java.awt.Frame dashboard) {
        initComponents();
        this.Dashboard = dashboard;
        this. Invoice = new invoice();
        loadEmployeeDetails();
    }
    private void loadEmployeeDetails(){
        try {
            ResultSet result = MySQL.search("SELECT * FROM `user`"
                + "INNER JOIN `status` ON `user`.`status_id` = `status`.`id`"
                + "INNER JOIN `type` ON `user`.`type_id` = `type`.`id` WHERE
`type`='CUSTOMER'"
            );
            DefaultTableModel          tableModel          =          (DefaultTableModel)
customerTable.getModel();
            tableModel.setRowCount(0);
            int count = 0;
            while (result.next()) {
                count++;
                Vector<String>customerList = new Vector();
                customerList.add(String.valueOf(count));
                customerList.add(result.getString("first_name")+"
"+result.getString("last_name"));
                customerList.add(result.getString("email"));
                ResultSet contacts = MySQL.search("SELECT * FROM `mobile` "
                    + "WHERE `user_id`='"+result.getString("id")+"'");
                String mobile="";
                boolean bool = true;
                while(contacts.next()){
                    if(bool){
                        mobile=contacts.getString("mobile");
                        bool = false;
                    }else{
                        mobile+=" / "+contacts.getString("mobile");
                    }
                }
            }
            customerList.add(mobile);
            customerList.add(result.getString("name"));
            tableModel.addRow(customerList);
        }
    }
}

```

```

    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void roundButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    SimpleDateFormat dateFormatter = new SimpleDateFormat
("yyyy-MM-dd HH:mm:ss");
    String fname = firstNameInput.getText();
    String lname = lastNameInput.getText();
    String mobileOne = mobileOneInput.getText();
    String mobileTwo = mobileTwoInput.getText();
    String email = emailInput.getText();

    customerBean cb = new customerBean();
    cb.setFirstName(fname);
    cb.setLastName(lname);
    cb.setMobileOne(mobileOne);

    if(nameValidation(fname)){
        return;
    }
    if(nameValidation(lname)){
        return;
    }
    if(mobileOne.isEmpty()){
        Notifications.getInstance().show
        (Notifications.Type.WARNING,
        Notifications.Location.TOP_RIGHT,
        "Mobile number is required!");
    }else if(mobileValidation(mobileOne)){
        return;
    }
    if(!mobileTwo.isEmpty()){
        if(mobileValidation(mobileTwo)){
            return;}
    }
    if(emailValidation(email)){
        return;
    }
    try {
        ResultSet resultSet = MySQL.search("SELECT * FROM `user` "
        + "WHERE `email`='"+email+"'");
        if(resultSet.next()){
            cb.setId(resultSet.getString("id"));
            JOptionPane.showMessageDialog
            (this, "Customer already registered", "Warning",
            JOptionPane.WARNING_MESSAGE);
            Invoice.setCustomerBean(cb);
        }
    }
}

```

```

        Invoice.setVisible(true);
        Invoice.setLocationRelativeTo(this);
    }else{
        String joinedDate = dateFormatter.format(new Date());
        MySQL.iud("INSERT INTO `user` (`first_name`, `last_name`, `email`, "
            + " `status_id`, `type_id`, `joined_date`) " +
            "VALUES ('" + fname + "', '" + lname + "', '" + email + "', "
            + " '1', '3', '" + joinedDate + "')");
        ResultSet contactDetails = MySQL.search("SELECT `id` FROM `user`"
            + " WHERE `email`='"+email+"'");
        if(contactDetails.next()){
            cb.setId(contactDetails.getString("id"));
            Invoice.setCustomerBean(cb);
            String userId = contactDetails.getString("id");
            MySQL.iud("INSERT INTO `mobile` (`mobile`, `user_id`) "
                + "VALUES ('"+mobileOne+"', '"+userId+"')");
            if(!mobileTwo.isEmpty()){
                MySQL.iud("INSERT INTO `mobile` (`mobile`, `user_id`) "
                    + " VALUES ('"+mobileTwo+"', '"+userId+"')");
            }
            JOptionPane.showMessageDialog(this,
                "Customer registration successfull", "Success",
                JOptionPane.INFORMATION_MESSAGE);
        }
        Invoice.setVisible(true);
    }

} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}
}

private boolean nameValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance().
            show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "This field is required!");
        return true;
    }else{
        return false;
    }
}

private boolean mobileValidation(String value){
    if(!value.matches(Validation.MOBILE_VALIDATION.validate())){
        Notifications.getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Provide valid mobile number");
    }
}

```



```

        return true;
    }
    return false;
}
private boolean emailValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Email is required");
        return true;
    }else if(!value.matches(Validation.EMAIL_VALIDATION.validate())){
        Notifications.getInstance().
            show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Provide valid email address");
        return true;
    }
    return false;
}
private void roundButton7ActionPerformed(java.awt.event.ActionEvent evt) {
    String text = searchInput.getText();
    if(!text.isBlank()){
        try{
            ResultSet result = MySQL.search("SELECT * FROM `user` INNER JOIN `type` "
                + "ON `user`.`type_id`=`type`.`id` INNER JOIN `status` "
                + "ON `user`.`status_id`=`status`.`id`"
                + "WHERE (`first_name` LIKE '"+text+"%' "
                + "OR `email` LIKE '"+text+"%') AND `type`='CUSTOMER'");
            DefaultTableModel                                tableModel                                =
            (DefaultTableModel)customerTable.getModel();
            tableModel.setRowCount(0);
            int count =0;
            while(result.next()){
                Vector<String> search = new Vector();
                count++;
                search.add(String.valueOf(count));
                search.add(result.getString("first_name")+" "+result.getString("last_name"));
                search.add(result.getString("email"));
                ResultSet mobileResult = MySQL.search("SELECT * FROM `mobile` "
                    + "WHERE `user_id`='"+result.getString("id")+"'");
                boolean bool = true;
                String mobile = "";
                while(mobileResult.next()){
                    if(bool){
                        mobile = mobileResult.getString("mobile");
                        bool = false;
                    }else{

```

```

        mobile+=" / "+mobileResult.getString("mobile");
    }
}
search.add(mobile);
search.add(result.getString("name"));
tableModel.addRow(search);
}
}catch (SQLException e){
e.printStackTrace();
}
}else{
loadEmployeeDetails();
}
}
private void customerTableMouseClicked(java.awt.event.MouseEvent evt) {
    if(evt.getClickCount()==2){
        int selectedRow = customerTable.getSelectedRow();
        String fullName= (String)(customerTable.getValueAt(selectedRow, 1));
        String email = (String)customerTable.getValueAt(selectedRow,2);
        String mobile = (String)customerTable.getValueAt(selectedRow,3);
        String status = (String) (customerTable.getValueAt(selectedRow, 4));
        String type="";
        try {
            ResultSet result = MySQL.search("SELECT `type` FROM `type` "
                + "WHERE `id`=(SELECT `type_id` FROM `user` "
                + "WHERE `email`='"+email+"')");
            if(result.next()){
                type = result.getString("type");
                System.out.println(type);
            }
        } catch (SQLException e) {
            Splash.logger.warning(e.getMessage());
        }
        String[] split = fullName.split(" ");
        String firstName = split[0];
        String lastName = split[1];

        String mobileOne = "";
        String mobileTwo = "";

        if(mobile.contains("/")){
            String[] Mobile = mobile.split(" / ");
            mobileOne = Mobile[0];
            mobileTwo = Mobile[1];
        }else{
            mobileOne = mobile;
        }
        CustomerDialog customersDialog = new CustomerDialog(Dashboard, true);
    }
}

```

```

customersDialog.setLocationRelativeTo(Dashboard);

customersDialog.getFirstNameInput().setText(firstName);
customersDialog.getLastNameInput().setText(lastName);
customersDialog.getEmailInput().setText(email);
customersDialog.getMobileOneInput().setText(mobileOne);
customersDialog.getMobileTwoInput().setText(mobileTwo);
customersDialog.getStatusCombo().setSelectedItem(status);
customersDialog.getUserTypeCombo().setSelectedItem(type);
customersDialog.setVisible(true);
}
}
private void mobileOneInputKeyReleased(java.awt.event.KeyEvent evt) {
    String inputText = mobileOneInput.getText();
    if (!inputText.isEmpty() && inputText.length() == 10) {
        try {
            ResultSet result = MySQL.search("SELECT * FROM `mobile` "
                + "WHERE `mobile`='" + inputText + "'");
            if (result.next()) {
                String userId = result.getString("user_id");
                ResultSet details = MySQL.search("SELECT * FROM `user` "
                    + "WHERE `id`='" + userId + "' AND `type_id`='3'");
                if (details.next()) {
                    firstNameInput.setText(details.getString("first_name"));
                    lastNameInput.setText(details.getString("last_name"));
                    emailInput.setText(details.getString("email"));
                }
            }
        } catch (SQLException e) {
            Splash.logger.warning(e.getMessage());
        }
    }
}
private void searchInputKeyReleased(java.awt.event.KeyEvent evt) {
    String text = searchInput.getText();
    if (!text.isBlank()){
        try{
            ResultSet result = MySQL.search("SELECT * FROM `user` INNER JOIN `type` "
                + "ON `user`.`type_id`=`type`.`id` INNER JOIN `status` "
                + "ON `user`.`status_id`=`status`.`id` "
                + "WHERE (`first_name` LIKE '"+text+"%' "
                + "OR `email` LIKE '"+text+"%') AND `type`='CUSTOMER'");
            DefaultTableModel tableModel =
            (DefaultTableModel)customerTable.getModel();
            tableModel.setRowCount(0);
            int count =0;
            while(result.next()){
                Vector<String> search = new Vector();

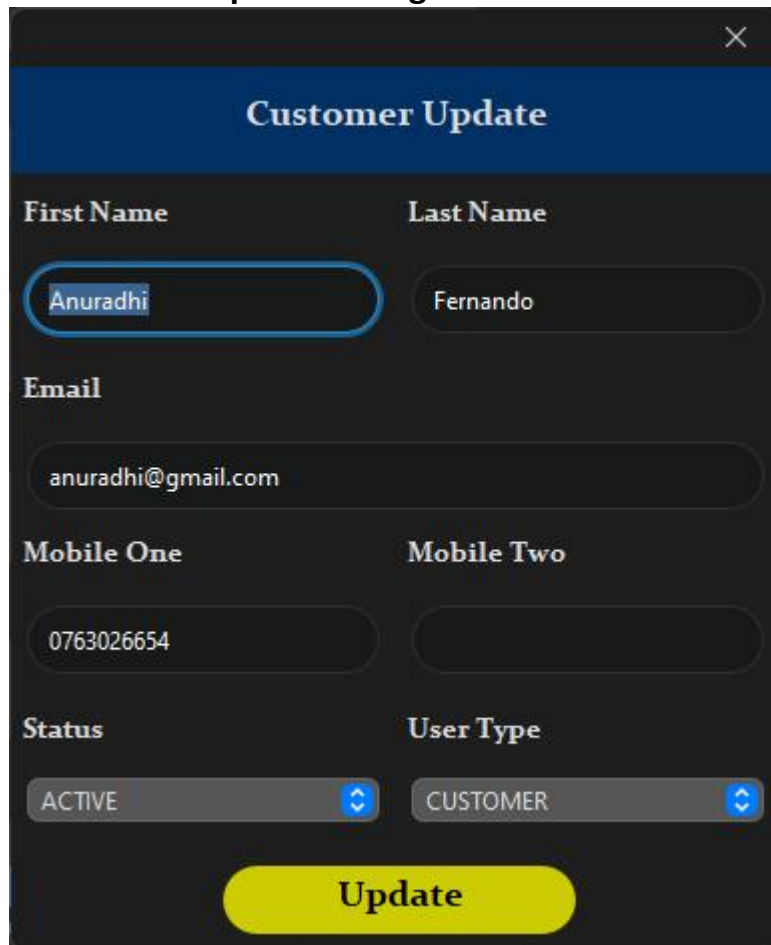
```

```

count++;
search.add(String.valueOf(count));
search.add(result.getString("first_name")+" "+result.getString("last_name"));
search.add(result.getString("email"));
ResultSet mobileResult = MySQL.search("SELECT * FROM `mobile` "
    + "WHERE `user_id`='"+result.getString("id")+"'");
boolean bool = true;
String mobile = "";
while(mobileResult.next()){
    if(bool){
        mobile = mobileResult.getString("mobile");
        bool = false;
    }else{
        mobile+=" / "+mobileResult.getString("mobile");
    }
}
search.add(mobile);
search.add(result.getString("name"));
tableModel.addRow(search);
}
}catch (SQLException e){
e.printStackTrace();
}
}else{
loadEmployeeDetails();
}
}

```

3.8. Customer Update Dialog



The image shows a 'Customer Update' dialog box with a dark theme. It contains several input fields and dropdown menus. The 'First Name' field is highlighted with a red border and contains the text 'Anuradhi'. The 'Last Name' field contains 'Fernando'. The 'Email' field contains 'anuradhi@gmail.com'. The 'Mobile One' field contains '0763026654'. The 'Mobile Two' field is empty. The 'Status' dropdown menu is set to 'ACTIVE' and the 'User Type' dropdown menu is set to 'CUSTOMER'. A large orange 'Update' button is at the bottom.

Code

```
import com.formdev.flatlaf.themes.FlatMacDarkLaf;
import javax.swing.JComboBox;
import lk.teveram.kuweni.component.RoundTextField;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Vector;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JOptionPane;
import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.gui.Splash;
import lk.teveram.kuweni.validation.Validation;
import raven.toast.Notifications;
public class CustomerDialog extends javax.swing.JDialog {
    public CustomerDialog(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        updateUserStatusCombo();
        userTypeLoad();
    }
    private void updateUserStatusCombo() {
        try {
            ResultSet result = MySQL.search("SELECT * FROM `status`");
```

```

        Vector<String> statusList = new Vector();
        statusList.add("Select status");
        while (result.next()) {
            statusList.add(result.getString("name"));
        }
        DefaultComboBoxModel comboBoxMode = new
DefaultComboBoxModel(statusList);
        this.statusCombo.setModel(comboBoxMode);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void userTypLoad(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `type`");
        Vector <String> typeList = new Vector();
        typeList.add("Select type");
        while(result.next()){
            typeList.add(result.getString("type"));
        }
        DefaultComboBoxModel comboBoxModel = new
DefaultComboBoxModel(typeList);
        userTypeCombo.setModel(comboBoxModel);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void updateButtonActionPerformed(java.awt.event.ActionEvent evt) {
    String firstName = firstNameInput.getText();
    String lastName = lastNameInput.getText();
    String email = emailInput.getText();
    String mobileOne = mobileOneInput.getText();
    String mobileTwo = mobileTwoInput.getText();
    int type = userTypeCombo.getSelectedIndex();
    int status = statusCombo.getSelectedIndex();
    System.out.println(status);
    if(nameValidation(firstName)){
        return;
    }
    if(nameValidation(lastName)){
        return;
    }
    if(emailValidation(email)){
        return;
    }
    if (mobileOne.isEmpty()) {
        Notifications
            .getInstance()

```

```

        .show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT,
            "Mobile one is required!");
    } else {
        if (mobileValidation(mobileOne)) {
            return;
        }
    }
    if (!mobileTwo.isEmpty()) {
        if (mobileValidation(mobileTwo)) {
            return;
        }
    }
    if (type < 1) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Select employee type!");
        return;
    }
    if (status < 1) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Select employee status!");
        return;
    }
    MySQL.iud("UPDATE `user` INNER JOIN `mobile` SET
`user`.`first_name`='"+firstName+"', "
        + "`user`.`last_name`='"+lastName+"', `user`.`type_id`='"+type+"', "
        + "`user`.`status_id`='"+status+"' WHERE `user`.`email`='"+email+"'");
    try {
        ResultSet currentUser = MySQL.search("SELECT `id` FROM `user` WHERE
`email`='"+ email + "'");
        String userId="";
        if (currentUser.next()){
            userId = currentUser.getString("id");
        }
        ResultSet contacts = MySQL.search("SELECT * FROM mobile "
            + "WHERE user_id='"+userId+"'");
        byte increment = 0;
        while(contacts.next()){
            MySQL.iud("UPDATE `mobile` SET `mobile`='"+mobileOne+" "
                + "WHERE `user_id`='"+userId+"'");
            if(increment > 0){
                MySQL.iud("UPDATE `mobile` SET `mobile`='"+mobileTwo+"

```

```

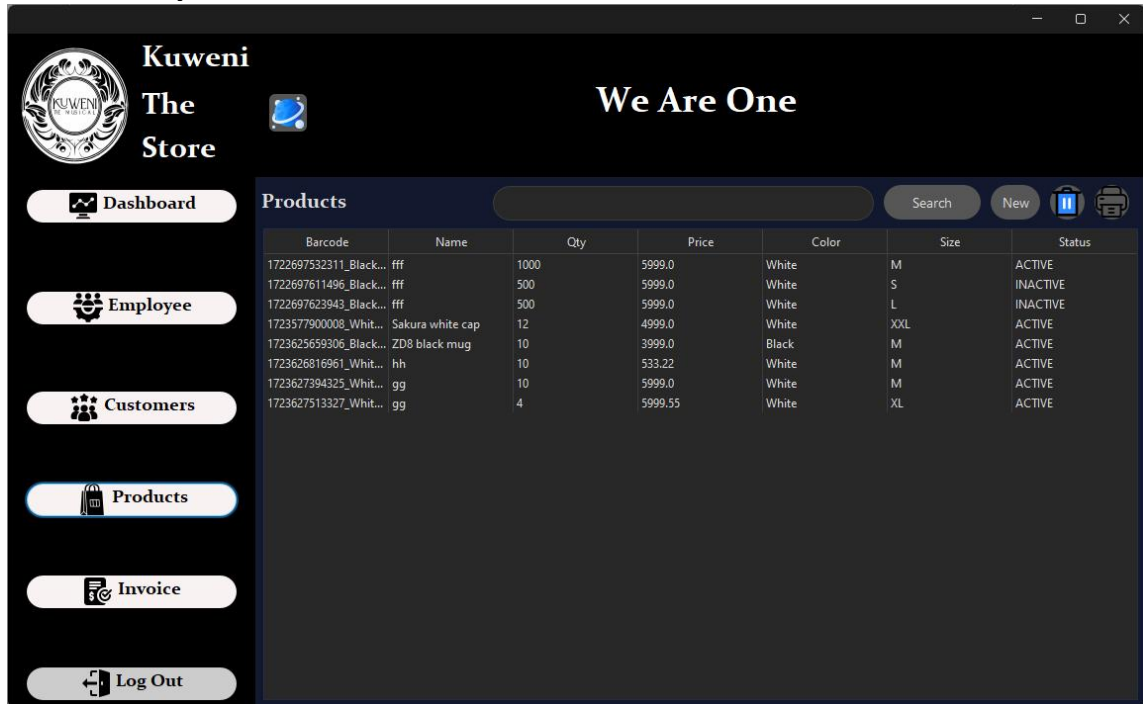
        + " WHERE `user_id`='"+userId+"'");
    }
    increment++;
}
} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}
JOptionPane.showMessageDialog(this, "Update successful!", "Warning",
    JOptionPane.WARNING_MESSAGE);
this.dispose();
}
private boolean nameValidation(String value){
    if(value.isEmpty()){
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT,
            "This field is requierd!");
        return true;
    }
    return false;
}
private boolean emailValidation(String value) {
    if (value.isEmpty()) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Email Addresss is required");
        return true;
    } else if (!value.matches
        (Validation.EMAIL_VALIDATION.validate())) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Provide valid Email Address");
        return true;
    }
    return false;
}
private boolean mobileValidation(String value) {
    if (!value.matches(Validation.MOBILE_VALIDATION.validate())) {
        Notifications
            .getInstance()
            .show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT,
                "Provide valid mobile number");
        return true;
    }
}

```



```
        return false;
    }
    public RoundTextField getFirstNameInput() {
        return firstNameInput;
    }
    public RoundTextField getLastNameInput() {
        return lastNameInput;
    }
    public RoundTextField getEmailInput() {
        return emailInput;
    }
    public RoundTextField getMobileOneInput() {
        return mobileOneInput;
    }
    public RoundTextField getMobileTwoInput() {
        return mobileTwoInput;
    }
    public JComboBox<String> getStatusCombo() {
        return statusCombo;
    }
    public JComboBox<String> getUserTypeCombo() {
        return userTypeCombo;
    }
}
```

3.9. Product panel



Description

This frame shows all the details of stock. In this frame, user can add new product and stock or update existing product quantity. If user wants to add new product, user can press new button and add product with stock quantity. When user types on the search bar, it searches invoice details from invoice id or customer name and loads the results to the table. With the print button, user can generate report of the product stock. And with the delete button, user can delete stock.

Code

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import lk.teveram.kuweni.connection.MySQL;
import java.util.Vector;
import javax.swing.table.DefaultTableModel;
import lk.teveram.kuweni.dialog.ProductDialog;
import lk.teveram.kuweni.gui.Splash;
import net.sf.jasperreports.engine.JRDataSource;
import net.sf.jasperreports.engine.JRException;
import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.data.JRTableModelDataSource;
import net.sf.jasperreports.view.JasperViewer;
```

```
public class Products extends javax.swing.JPanel {
    private final java.awt.Frame Dashboard;
    private final SimpleDateFormat sdf;
```

```

public Products(java.awt.Frame dashboard) {
    initComponents();
    this.Dashboard = dashboard;
    this.sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
    loadTable();
}

public void loadTable(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `stock` "
            + "INNER JOIN `size` ON `stock`.`size_id`=`size`.`id`"
            + "INNER JOIN `color` ON `color`.`id`=`stock`.`color_id`"
            + "INNER JOIN `status` ON `status`.`id`=`stock`.`status_id`");
        DefaultTableModel tableModel = (DefaultTableModel)productTable.getModel();
        tableModel.setRowCount(0);
        int count=0;
        while(result.next()){
            count++;
            Vector<String> productList= new Vector();
            productList.add(result.getString("barcode"));
            productList.add(result.getString("name"));
            productList.add(result.getString("qty"));
            productList.add(result.getString("selling_price"));
            productList.add(result.getString("color"));
            productList.add(result.getString("size"));
            productList.add(result.getString("status.name"));
            tableModel.addRow(productList);
        }
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void newButtonActionPerformed(java.awt.event.ActionEvent evt) {
    ProductDialog dialog = new ProductDialog(Dashboard,true);
    dialog.setLocationRelativeTo(Dashboard);
    dialog.setProductPanel(this);
    dialog.setVisible(true);
}

private void roundButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    if(productTable.getSelectedRowCount()==1){
        int row = productTable.getSelectedRow();
        String barcode =(String) productTable.getValueAt(row,0);
        MySQL.iud("DELETE FROM `stock` WHERE `barcode`='"+barcode+"'");
        loadTable();
    }
}

private void roundButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    Map<String, Object> parameters = new HashMap<>();
    String format = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new

```

```

Date());
parameters.put("received", null);
String filePath = "C:\\Users\\tevin\\OneDrive\\Documents\\NetBeansProjects"
    + "\\Kuweni last\\src\\resource\\kuweni_product_report.jasper";
try {
    JRDataSource dataSource = new
JTableModelDataSource(productTable.getModel());
    JasperPrint jasperPrint = JasperFillManager.fillReport(filePath, parameters,
dataSource);
    JasperViewer.viewReport(jasperPrint, false);
} catch (JRException e) {
Splash.logger.warning(e.getMessage());
}
}

private void searchInputKeyReleased(java.awt.event.KeyEvent evt) {
String text = searchInput.getText();
if(!text.isBlank()){
try{
    ResultSet result = MySQL.search("SELECT * FROM `stock` "
        + "INNER JOIN `color` ON `color`.`id`=`stock`.`color_id`"
        + "INNER JOIN `size` ON `size`.`id`=`stock`.`size_id`"
        + "INNER JOIN `status` ON `status`.`id`=`stock`.`status_id`"
        + "WHERE `stock`.`name` LIKE '"+text+"%'");
    DefaultTableModel tableModel = (DefaultTableModel)productTable.getModel();
    tableModel.setRowCount(0);
    while(result.next()){
        Vector<String> vector = new Vector();
        vector.add(result.getString("barcode"));
        vector.add(result.getString("stock.name"));
        vector.add(result.getString("qty"));
        vector.add(result.getString("selling_price"));
        vector.add(result.getString("color"));
        vector.add(result.getString("size"));
        vector.add(result.getString("status.name"));
        tableModel.addRow(vector);
    }
}catch(SQLException e){
    Splash.logger.warning(e.getMessage());
}
}else{
    loadTable();
}
}

private void roundButton2ActionPerformed(java.awt.event.ActionEvent evt) {
String text = searchInput.getText();
if(!text.isBlank()){
try{
    ResultSet result = MySQL.search("SELECT * FROM `stock` "

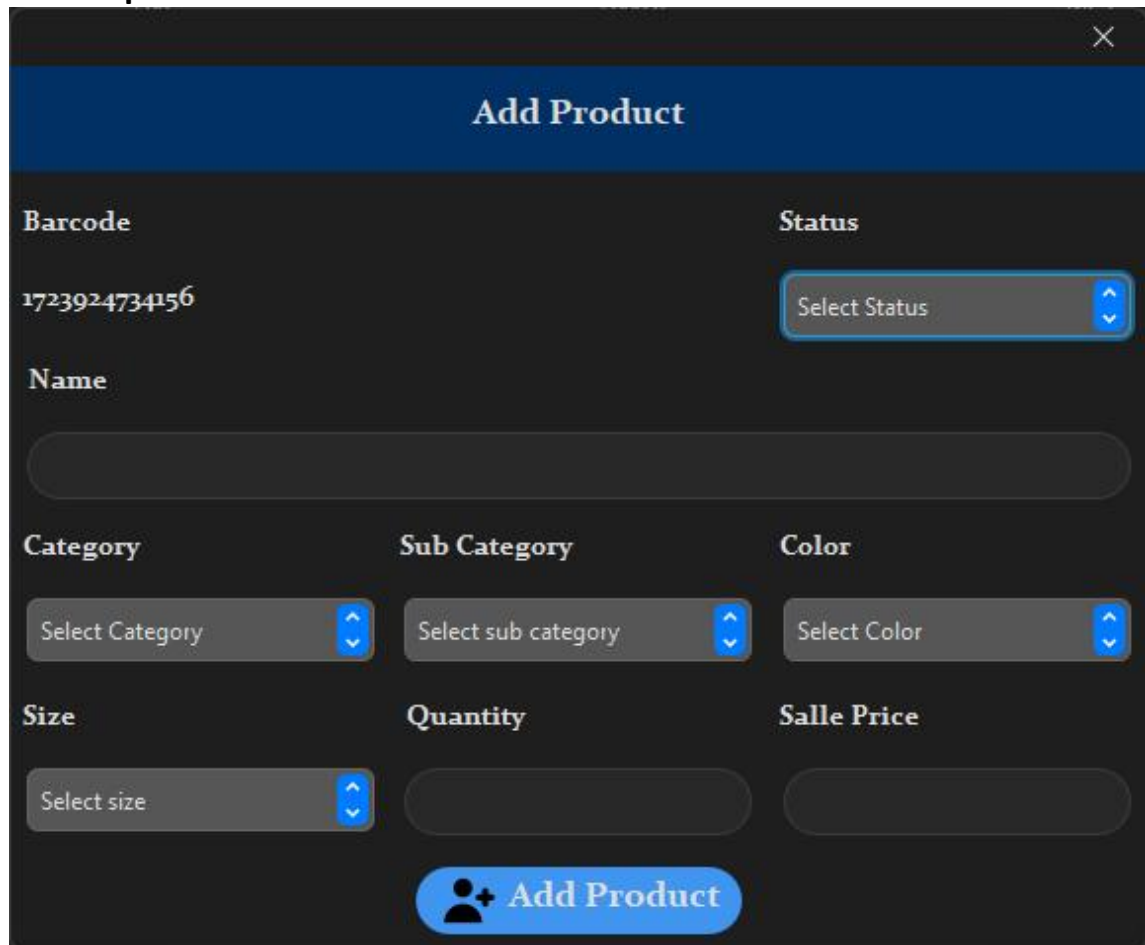
```

```

        + "INNER JOIN `color` ON `color`.`id`=`stock`.`color_id`"
        + "INNER JOIN `size` ON `size`.`id`=`stock`.`size_id`"
        + "INNER JOIN `status` ON `status`.`id`=`stock`.`status_id`"
        + "WHERE `stock`.`name` LIKE '"+text+"%'"");
DefaultTableModel tableModel = (DefaultTableModel)productTable.getModel();
tableModel.setRowCount(0);
while(result.next()){
    Vector<String> vector = new Vector();
    vector.add(result.getString("barcode"));
    vector.add(result.getString("stock.name"));
    vector.add(result.getString("qty"));
    vector.add(result.getString("selling_price"));
    vector.add(result.getString("color"));
    vector.add(result.getString("size"));
    vector.add(result.getString("status.name"));
    tableModel.addRow(vector);
}
}catch(SQLException e){
    Splash.logger.warning(e.getMessage());
}
}else{
    loadTable();
}
}

```

3.10. Add product frame

A dark-themed dialog box titled "Add Product" with a close button (X) in the top right corner. The form contains several input fields: "Barcode" with the value "1723924734156", "Status" with a dropdown menu showing "Select Status", "Name" with a text input field, "Category" with a dropdown menu showing "Select Category", "Sub Category" with a dropdown menu showing "Select sub category", "Color" with a dropdown menu showing "Select Color", "Size" with a dropdown menu showing "Select size", "Quantity" with a text input field, and "Salle Price" with a text input field. At the bottom, there is a blue button with a plus icon and the text "Add Product".

Description

In this frame, user can add new products, All the input fields are validated. When user select product size and color, bar code will regenerate. If user add existing product, it will be update the quantity of the existing product.

Code

```
import com.formdev.flatlaf.FlatDarkLaf;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.Vector;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JOptionPane;
import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.gui.Splash;
import lk.teveram.kuweni.panel.Products;
import raven.toast.Notifications;
```

```
public class ProductDialog extends javax.swing.JDialog {
    private String barCodeNo;
    private Map <String, Integer> statusMap;
```

```

private Map <String, Integer> colorMap;
private Map <String, Integer> categoryMap;
private Map <String, Integer> subCategoryMap;
private Map <String, Integer> sizeMap;
private Products products;
public ProductDialog(java.awt.Frame parent, boolean modal) {
    super(parent, modal);
    initComponents();
    this.statusMap = new HashMap<>();
    this.colorMap = new HashMap<>();
    this.categoryMap = new HashMap<>();
    this.subCategoryMap = new HashMap<>();
    this.sizeMap = new HashMap<>();
    loadStatus();
    loadColor();
    loadCategory();
    loadSubCategory();
    loadSize();
    barCodeGenerate();
}
public void setProductPanel(Products pp){
    this.products=pp;
}
private void barCodeGenerate() {
    String selectedSize = (String) sizeCombo.getSelectedItemAt();
    String selectedColor = (String) colorCombo.getSelectedItemAt();
    this.barCodeNo = String.valueOf(System.currentTimeMillis());
    if (!selectedSize.equals("Select size") && !selectedColor.equals("Select Color")) {
        this.barCodeNo += "_" + selectedColor + "_" + selectedSize;
    }
    this.barCodeLabel.setText(this.barCodeNo);
}
private void loadStatus(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `status`");
        Vector<String> status = new Vector<>();
        status.add("Select Status");
        while(result.next()){
            status.add(result.getString("name"));
            statusMap.put(result.getString("name"), result.getInt("id"));
        }
        DefaultComboBoxModel comboBoxModel = new
DefaultComboBoxModel(status);
        statusCombo.setModel(comboBoxModel);
    } catch (Exception e) {
        Splash.logger.warning(e.getMessage());
    }
}
}

```

```

private void loadColor(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `color`");
        Vector<String>color = new Vector<>();
        color.add("Select Color");
        while(result.next()){
            color.add(result.getString("color"));
            colorMap.put(result.getString("color"),result.getInt("id"));
        }
        DefaultComboBoxModel comboBoxModel = new DefaultComboBoxModel(color);
        colorCombo.setModel(comboBoxModel);
    } catch (Exception e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void loadCategory(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `category`");
        Vector<String>category = new Vector<>();
        category.add("Select Category");
        while(result.next()){
            category.add(result.getString("category"));
            categoryMap.put(result.getString("category"),result.getInt("id"));
        }
        DefaultComboBoxModel ComboModel = new DefaultComboBoxModel(category);
        categoryCombo.setModel(ComboModel);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void loadSubCategory(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `sub_category`");
        Vector<String>subCategory = new Vector();
        subCategory.add("Select sub category");
        while(result.next()){
            subCategory.add(result.getString("sub_category"));
            subCategoryMap.put(result.getString("sub_category"), result.getInt("id"));
        }
        DefaultComboBoxModel ComboModel = new
DefaultComboBoxModel(subCategory);
        subCategoryCombo.setModel(ComboModel);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void loadSize(){
    try {

```



```

        ResultSet result = MySQL.search("SELECT * FROM `size`");
        Vector<String>size = new Vector();
        size.add("Select size");
        while(result.next()){
            size.add(result.getString("size"));
            sizeMap.put(result.getString("size"),result.getInt("id"));
        }
        DefaultComboBoxModel comboModel = new DefaultComboBoxModel(size);
        sizeCombo.setModel(comboModel);
    } catch (SQLException e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void roundButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    SimpleDateFormat dateFormatter = new SimpleDateFormat("yyyy:MM:dd
HH:mm:ss");
    String productName = productNameInput.getText().trim();
    if (productName.isEmpty()) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please enter product name");
        return;
    }
    Integer categoryId = categoryMap.get((String) categoryCombo.getSelectedItem());
    Integer colorId = colorMap.get((String) colorCombo.getSelectedItem());
    Integer subCategoryId = subCategoryMap.get((String)
subCategoryCombo.getSelectedItem());
    Integer sizeId = sizeMap.get((String) sizeCombo.getSelectedItem());
    Integer statusId = statusMap.get((String) statusCombo.getSelectedItem());
    String qty = qtyInput.getText();
    String priceText = priceInput.getText();
    if (categoryId == null) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please select a valid category");
        return;
    }
    if (subCategoryId == null) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please select a valid subcategory");
        return;
    }
    if (colorId == null) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please select a valid color");
        return;
    }
    if (sizeId == null) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please select a valid size");
    }
}

```

```

        return;
    }
    if (qty.isEmpty()) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Quantity cannot be empty.");
        return;
    }
    int quantity;
    try {
        quantity = Integer.parseInt(qty);
        if (quantity <= 0) {
            Notifications.getInstance().show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT, "Please enter a valid quantity !");
            return;
        }
    } catch (NumberFormatException e) {
        Splash.logger.warning(e.getMessage());
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please enter a valid quantity !");
        return;
    }
    double sellingPrice;
    if (priceText.isEmpty()) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Selling price cannot be empty !");
        System.out.println("return");
        return;
    }
    if (twoDecimalPlaces(priceText)) {
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please enter a valid price !");
        return;
    }
    try {
        sellingPrice = Double.parseDouble(priceText);
        if (sellingPrice <= 0) {
            Notifications.getInstance().show(Notifications.Type.WARNING,
                Notifications.Location.TOP_RIGHT, "Please enter a valid price !");
            return;
        }
    } catch (NumberFormatException e) {
        Splash.logger.warning(e.getMessage());
        Notifications.getInstance().show(Notifications.Type.WARNING,
            Notifications.Location.TOP_RIGHT, "Please enter a valid price !");
        return;
    }
    if (statusId == null) {
        Notifications.getInstance().show(Notifications.Type.WARNING,

```

```

        Notifications.Location.TOP_RIGHT, "Please select a valid status");
    return;
}
String productId = String.valueOf(System.currentTimeMillis());

try {
    ResultSet productResult = MySQL.search("SELECT * FROM `product` "
        + "WHERE `category_id`='" + categoryId + "' "
        + "AND `sub_category_id`='" + subCategoryId + "'");
    if (productResult.next()) {
        String existProductId = productResult.getString("id");
        ResultSet stockResult = MySQL.search("SELECT * FROM `stock` "
            + "WHERE `product_id`='" + existProductId + "' "
            + "AND `color_id`='" + colorId + "' "
            + "AND `size_id`='" + sizeId + "' "
            + "AND `category_id`='" + categoryId + "' AND "
            + "`sub_category_id`='" + subCategoryId + "' AND "
            + "`selling_price`='" + sellingPrice + "'");
        if (stockResult.next()) {
            int existingQty = stockResult.getInt("qty");
            int newQty = existingQty + quantity;
            MySQL.iud("UPDATE `stock` SET `qty`='" + newQty + "' "
                + "WHERE `barcode`='" + stockResult.getString("barcode") + "'");
            JOptionPane.showMessageDialog(this, "Stock updated with new quantity: " +
newQty,
                "Information", JOptionPane.INFORMATION_MESSAGE);
        } else {
            System.out.println("No existing stock found, ");
            String date = dateFormatter.format(new Date());
            MySQL.iud("INSERT INTO `stock` (`barcode`, `name`, `qty`, "
                + "`selling_price`, `added_date`, `product_id`, `color_id`, "
                + "`category_id`, `sub_category_id`, `size_id`, `status_id`) "
                + "VALUES ('" + barCodeLabel.getText() + "','" + productName + "','"
                + "" + quantity + "','" + sellingPrice + "','" + date + "','" + existProductId +
"",
                + "" + colorId + "','" + categoryId + "','" + subCategoryId + "','"
                + "" + sizeId + "','" + statusId + "')");
            JOptionPane.showMessageDialog(this, "Added new stock",
                "Information", JOptionPane.INFORMATION_MESSAGE);
        }
    } else {
        String date = dateFormatter.format(new Date());
        MySQL.iud("INSERT INTO `product` (`id`, `name`, `category_id`, `sub_category_id`)
"
            + "VALUES('" + productId + "','" + productName + "','" + categoryId + "','" +
subCategoryId + "')");
        MySQL.iud("INSERT INTO `stock` (`barcode`, `name`, `qty`, `selling_price`, "
            + "`added_date`, `product_id`, `color_id`, `category_id`, "

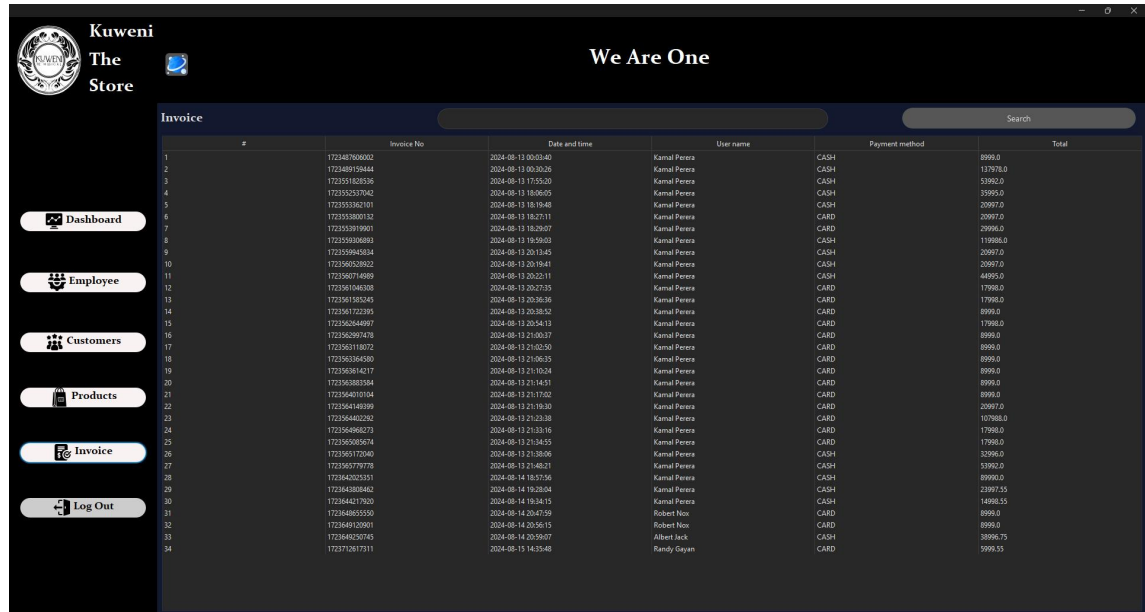
```

```

        + "`sub_category_id`,`size_id`,`status_id`) VALUES (" +
barCodeLabel.getText() + ", "
        + "" + productName + ", " + quantity + ", " + sellingPrice + ", "
        + "" + date + ", " + productId + ", " + colorId + ", "
        + "" + categoryId + ", " + subCategoryId + ", "
        + "" + sizeId + ", " + statusId + "));
OptionPane.showMessageDialog(this, "Product added successfully",
    "Information", JOptionPane.INFORMATION_MESSAGE);
this.dispose();
this.produts.loadTable();
    }
} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}
}
private boolean twoDecimalPlaces(String text) {
if (text.contains(".")) {
    String[] parts = text.split("//.");
    if (parts.length > 1 && parts[1].length() > 2) {
        return true;
    }
}
return false;
}
private void subCategoryComboboxItemStateChanged(java.awt.event.ItemEvent evt) {
}
private void colorComboboxItemStateChanged(java.awt.event.ItemEvent evt) {
    barCodeNo = null;
    barCodeGenerate();
}
private void sizeComboboxItemStateChanged(java.awt.event.ItemEvent evt) {
    barCodeNo = null;
    barCodeGenerate();
}
}

```

3.11. Invoice view



#	Invoice No	Date and time	User name	Payment method	Total
1	172348760002	2024-08-13 00:03:40	Kamali Perera	CASH	8999.0
2	172348915844	2024-08-13 00:20:26	Kamali Perera	CASH	13720.0
3	172351828536	2024-08-13 17:55:20	Kamali Perera	CASH	53982.0
4	172352537042	2024-08-13 18:06:05	Kamali Perera	CASH	35995.0
5	172353392101	2024-08-13 18:19:48	Kamali Perera	CASH	20997.0
6	172353399132	2024-08-13 18:27:11	Kamali Perera	CARD	20997.0
7	172353919901	2024-08-13 18:29:07	Kamali Perera	CARD	29998.0
8	172353930893	2024-08-13 18:59:03	Kamali Perera	CASH	119986.0
9	172359445834	2024-08-13 20:18:45	Kamali Perera	CASH	20997.0
10	172359528022	2024-08-13 20:19:41	Kamali Perera	CASH	20997.0
11	172360714989	2024-08-13 20:22:11	Kamali Perera	CASH	44995.0
12	172361048308	2024-08-13 20:27:35	Kamali Perera	CARD	17998.0
13	172361198340	2024-08-13 20:36:46	Kamali Perera	CARD	17998.0
14	172361723395	2024-08-13 20:38:52	Kamali Perera	CARD	8999.0
15	172362644997	2024-08-13 20:54:13	Kamali Perera	CARD	17998.0
16	172362997478	2024-08-13 21:00:37	Kamali Perera	CARD	8999.0
17	1723631118072	2024-08-13 21:02:50	Kamali Perera	CARD	8999.0
18	172363363450	2024-08-13 21:06:35	Kamali Perera	CARD	8999.0
19	172363614217	2024-08-13 21:10:24	Kamali Perera	CARD	8999.0
20	172363883384	2024-08-13 21:14:51	Kamali Perera	CARD	8999.0
21	172364071054	2024-08-13 21:17:62	Kamali Perera	CARD	8999.0
22	172364149399	2024-08-13 21:19:30	Kamali Perera	CARD	20997.0
23	172364402292	2024-08-13 21:23:38	Kamali Perera	CARD	107988.0
24	172364969273	2024-08-13 21:33:16	Kamali Perera	CARD	17998.0
25	172365069874	2024-08-13 21:34:52	Kamali Perera	CARD	17998.0
26	172365172040	2024-08-13 21:38:06	Kamali Perera	CASH	32998.0
27	172365779778	2024-08-13 21:48:21	Kamali Perera	CASH	53992.0
28	172364202331	2024-08-14 18:57:56	Kamali Perera	CASH	89990.0
29	172364308242	2024-08-14 19:20:54	Kamali Perera	CASH	23997.55
30	172364417920	2024-08-14 19:34:15	Kamali Perera	CASH	14998.55
31	172364865550	2024-08-14 20:47:59	Robert Noun	CARD	8999.0
32	1723649123601	2024-08-14 20:56:15	Robert Noun	CARD	8999.0
33	1723649123615	2024-08-14 20:56:07	Albert Jack	CASH	30996.75
34	1723726179311	2024-08-15 14:35:48	Randy Guyan	CARD	5998.55

Description

In this frame ,user can view all the invoice details.And user can search all the invoices by invoice id or customer name.

Code

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Vector;
import javax.swing.table.DefaultTableModel;
import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.gui.Splash;

public class Invoices extends javax.swing.JPanel {
    public Invoices() {
        initComponents();
        loadTable();
    }
    private void loadTable(){
        try{
            ResultSet result = MySQL.search("SELECT * FROM `invoice` " +
            " INNER JOIN `user` ON `user`.`id`=`invoice`.`user_id`" +
            " INNER JOIN `payment_Method` ON
            `invoice`.`payment_id`=`payment_Method`.`id`");

            DefaultTableModel tableModel = (DefaultTableModel)invoiceTable.getModel();
            tableModel.setRowCount(0);
            int count =0;
            while(result.next()){
                Vector<String> search = new Vector();
                count++;
                search.add(String.valueOf(count));
                search.add(result.getString("invoice.id"));
                search.add(result.getString("date_time"));
            }
        }
    }
}
```

```

        search.add(result.getString("first_name")+" "+result.getString("last_name"));
        search.add(result.getString("payment_Method"));
        search.add(result.getString("total"));

        tableModel.addRow(search);
    }
} catch (SQLException e){
    Splash.logger.warning(e.getMessage());
}
}

private void searchInputKeyTyped(java.awt.event.KeyEvent evt) {
    String text = searchInput.getText();
    try{
        ResultSet result = MySQL.search("SELECT * FROM `invoice`" +
            "INNER JOIN `user` " +
            "  ON `user`.`id` = `invoice`.`user_id`" +
            "INNER JOIN `status` " +
            "  ON `user`.`status_id` = `status`.`id`" +
            "INNER JOIN `payment_Method` " +
            "  ON `invoice`.`payment_id` = `payment_Method`.`id`" +
            "WHERE `invoice`.`id` LIKE '"+text+"%' " +
            "  OR `user`.`first_name` LIKE '"+text+"%'");
        DefaultTableModel tableModel = (DefaultTableModel)invoiceTable.getModel();
        tableModel.setRowCount(0);
        int count =0;
        while(result.next()){
            Vector<String> search = new Vector();
            count++;
            search.add(String.valueOf(count));
            search.add(result.getString("invoice.id"));
            search.add(result.getString("date_time"));
            search.add(result.getString("first_name")+" "+result.getString("last_name"));
            search.add(result.getString("payment_Method"));
            tableModel.addRow(search);
        }
    } catch (SQLException e){

        Splash.logger.warning(e.getMessage());
    }
}

private void roundButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    String text = searchInput.getText();
    if(!text.isBlank()){
    }
}
}

```

3.12. Generate invoice

Generate Invoice

Barcode Product Name Color Size

Category Sub category Selling Price Available Quantity Purchasing Quantity

Barcode	Name	Selling price	Qty	Total
---------	------	---------------	-----	-------

Summary

Paymnt Method Select method

Total items 0

Amount (LKR) 0.00

Paid Amount (LKR)

Balance (LKR)

Print

Description

When the customer registration was done, directly open this frame and cursor focus to the bar code text field. If user enter the bar code and press enter, all the stock details of that bar code are loading to the labels. And cursor focus to quantity input field. When the user enter a qty and press enter key, the table will update with that details and update the total items, total amount of the summary. If user select card for payment method user can directly press print button and generate invoice. If the user select cash method, user need to enter paid amount and press enter key. Then the user can print invoice.

Code

```
import com.formdev.flatlaf.FlatDarkLaf;
import java.awt.event.KeyEvent;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.Vector;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import lk.teveram.kuweni.connection.MySQL;
import lk.teveram.kuweni.model.customerBean;
import net.sf.jasperreports.engine.JRDataSource;
import net.sf.jasperreports.engine.JRException;
```

```

import net.sf.jasperreports.engine.JasperFillManager;
import net.sf.jasperreports.engine.JasperPrint;
import net.sf.jasperreports.engine.data.JRTableModelDataSource;
import net.sf.jasperreports.view.JasperViewer;

public class invoice extends javax.swing.JFrame {
    private Map <String, Integer> paymentMethodMap;
    private String invoiceNo;
    private final SimpleDateFormat sdf;
    private customerBean customer;
    private String customerId;
    private String customerName;
    private String mobile;

    public invoice() {
        initComponents();
        this.paymentMethodMap = new HashMap<>();
        this.sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        loadPaymentMethods();
        methodComboEnable();
        changelInvoiceNo();
        printInvoice.setEnabled(false);
        qtyInput.setEnabled(false);
    }

    public void setCustomerBean(customerBean cb){
        this.customer = cb;
        this.customerName = customer.getFirstName()+" "+customer.getLastName();
        this.mobile =customer.getMobileOne();
        this.customerId = customer.getId();
    }

    private void changelInvoiceNo(){
        this.invoiceNo = String.valueOf(System.currentTimeMillis());
    }

    private void barcodeInputKeyReleased(java.awt.event.KeyEvent evt) {
        if(evt.getKeyCode() == KeyEvent.VK_ENTER){
            try {
                ResultSet result = MySQL.search("SELECT * FROM `stock`"
                    + "INNER JOIN `product` ON `stock`.`product_id`=`product`.`id`"
                    + "INNER JOIN `color` ON `stock`.`color_id`=`color`.`id`"
                    + "INNER JOIN `size` ON `stock`.`size_id`=`size`.`id`"
                    + "INNER JOIN `category` ON `product`.`category_id`=`category`.`id`"
                    + "INNER JOIN `sub_category` ON `product`.`sub_category_id`=`sub_category`.`id`"
                    + "WHERE `stock`.`barcode`='"+(barcodeInput.getText()+"')");
                if(result.next()){
                    productNameLabel.setText(result.getString("product.name"));
                }
            }
        }
    }
}

```



```

        colorLabel.setText(result.getString("color"));
        sizeLabel.setText(result.getString("size"));
        categoryLabel.setText(result.getString("category"));
        subCategoryLabel.setText(result.getString("sub_category"));
        sellingPriceLabel.setText(result.getString("stock.selling_price"));
        availableQtyLabel.setText(result.getString("stock.qty"));
        qtyInput.setEnabled(true);
        qtyInput.grabFocus();
    }
} catch (SQLException e) {
    Splash.logger.warning(e.getMessage());
}
}
}

private void qtyInputKeyReleased(java.awt.event.KeyEvent evt) {
    if(evt.getKeyCode()==KeyEvent.VK_ENTER){
        int avaQty = Integer.parseInt(availableQtyLabel.getText());
        int ordQty = Integer.parseInt(qtyInput.getText());
        if(avaQty >= ordQty && ordQty > 0){
            DefaultTableModel tableModel = (DefaultTableModel)invoiceTable.getModel();
            boolean isFound = false;
            int rowIndex = 0;
            for(int i = 0; i < tableModel.getRowCount(); i++){
                String valueAt =(String) invoiceTable.getValueAt(i, 0);
                if(valueAt.equals(barcodeInput.getText())){
                    isFound = true;
                    rowIndex = i;
                    break;
                }
            }
            if(isFound){
                String currentQty = String.valueOf(invoiceTable.getValueAt(rowIndex, 3));
                int fullQty = Integer.parseInt(currentQty) + ordQty;
                double total = Double.parseDouble(sellingPriceLabel.getText())* fullQty;
                invoiceTable.setValueAt(fullQty, rowIndex, 3);
                invoiceTable.setValueAt(total, rowIndex, 4);
            }else{
                Vector<Object> invoiceData = new Vector();
                invoiceData.add(barcodeInput.getText());
                invoiceData.add(productNameLabel.getText()+"
("+colorLabel.getText()+"_"+sizeLabel.getText()+")");
                invoiceData.add(Double.valueOf(sellingPriceLabel.getText()));
                invoiceData.add(ordQty);
                Double total = Double.parseDouble(sellingPriceLabel.getText()) * ordQty;
                invoiceData.add(total);
                tableModel.addRow(invoiceData);
            }
            barcodeInput.setText("");
        }
    }
}

```

```

        productNameLabel.setText("");
        colorLabel.setText("");
        sizeLabel.setText("");
        categoryLabel.setText("");
        subCategoryLabel.setText("");
        sellingPriceLabel.setText("");
        availableQtyLabel.setText("");
        qtyInput.setText("");
        qtyInput.setEnabled(false);
        barcodeInput.setEnabled(true);
        barcodeInput.grabFocus();
        updateSummary();
    }else{
        JOptionPane.showMessageDialog(this,"Please enter valid quantity",
            "Warning",JOptionPane.WARNING_MESSAGE);
    }
}
}
private void paidAmountInputKeyReleased(java.awt.event.KeyEvent evt) {
    if (evt.getKeyCode() == KeyEvent.VK_ENTER
    && !paidAmountInput.getText().isBlank()) {
        try {
            Double paidAmount = Double.parseDouble(paidAmountInput.getText());
            Double totalAmount = Double.parseDouble(totalAmountLabel.getText());
            if (paidAmount >= totalAmount && !twoDecimalPlaces(paidAmountInput.getText()))
            {
                Double balance = paidAmount - totalAmount;
                balanceLabel.setText(String.format("%.2f", balance));
                printInvoice.setEnabled(true);
            } else {
                JOptionPane.showMessageDialog(this, "Please enter a valid amount",
                    "Warning", JOptionPane.WARNING_MESSAGE);
            }
        } catch (NumberFormatException e) {
            Splash.logger.warning(e.getMessage());
            JOptionPane.showMessageDialog(this, "Invalid number format",
                "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
private boolean twoDecimalPlaces(String text) {
    if (text.contains(".")) {
        String[] parts = text.split("\\.");
        if (parts.length > 1 && parts[1].length() > 2) {
            return true;
        }
    }
    return false;
}

```

```

}
private void methodsComboBoxItemStateChanged(java.awt.event.ItemEvent evt) {
    methodComboEnable();
}

private void printInvoiceActionPerformed(java.awt.event.ActionEvent evt) {
    boolean isSuccess = updateInvoiceTable();
    if (isSuccess) {
        Map<String, Object> parameters = new HashMap<>();
        parameters.put("invoiceNo", invoiceNo);
        String format = sdf.format(new Date());
        parameters.put("date", format.split(" ")[0]);
        parameters.put("time", format.split(" ")[1]);
        parameters.put("customerName", customerName);
        parameters.put("customerMobile", mobile);
        parameters.put("paymentMethod", methodsComboBox.getSelectedItemAt());
        parameters.put("itemsTotal", itemsLabel.getText());
        parameters.put("totalPrice", totalAmountLabel.getText());
        parameters.put("balance", balanceLabel.getText());
        if (paidAmountInput.isEnabled() && !paidAmountInput.getText().isBlank()) {
            parameters.put("paidAmount", paidAmountInput.getText());
        } else {
            parameters.put("received", null);
        }
        String filePath = "C:\\Users\\Risinu Ramuditha\\Documents\\NetBeansProjects"
            + "\\Kuweni new\\src\\resource\\kuweni_invoice.jasper";
        JRDataSource dataSource = new
        JTableModelDataSource(invoiceTable.getModel());
        try {
            JasperPrint fillReport = JasperFillManager.fillReport(filePath, parameters,
            dataSource);
            JasperViewer.viewReport(fillReport, false);
            System.out.println(parameters);
        } catch (JRException ex) {
            Splash.logger.warning(ex.getMessage());
        }
    } else {
        JOptionPane.showMessageDialog(this, "Something went wrong! Please check it",
            "Warning Message", JOptionPane.WARNING_MESSAGE);
    }
}

private boolean updateInvoiceTable(){
    boolean isSuccess = false;

    int id =Integer.valueOf(customerId);
    Double total = Double.valueOf(totalAmountLabel.getText());
    int paymentId = this.paymentMethodMap.get((String)
    methodsComboBox.getSelectedItemAt());

```

```

try{
    MySQL.iud("INSERT INTO `invoice` "
        + "VALUES('"+invoiceNo+"','"+sdf.format(new
Date())+"','"+total+"','"+paymentId+"','"+id+"')");
    for(int i = 0; i<invoiceTable.getRowCount(); ++i){
        int qty = (Integer)invoiceTable.getValueAt(i, 3);
        String barcode = (String)invoiceTable.getValueAt(i, 0);
        System.out.println(barcode);
        System.out.println(barcode);
        System.out.println(i);
        System.out.println(qty);
        MySQL.iud("INSERT INTO `invoice_item`
VALUES('"+qty+"','"+barcode+"','"+invoiceNo+"')");
        ResultSet result = MySQL.search("SELECT `qty` FROM `stock` WHERE `barcode`=
 '"+barcode+"'");
        if(result.next()){
            int currentQty = result.getInt("qty");
            int newQty = currentQty-qty;
            MySQL.iud("UPDATE `stock` SET `qty` = '"+newQty+"' WHERE
`barcode`='"+barcode+"'");
            return true;
        }
    }
} catch(SQLException e){
    Splash.logger.warning(e.getMessage());
}
return isSuccess;
}

private void loadPaymentMethods(){
    try {
        ResultSet result = MySQL.search("SELECT * FROM `payment_Method`");
        Vector<String> status = new Vector<>();
        status.add("Select method");
        while(result.next()){
            status.add(result.getString("payment_method"));
            paymentMethodMap.put(result.getString("payment_method"),
result.getInt("id"));
        }
        DefaultComboBoxModel comboBoxModel = new
DefaultComboBoxModel(status);
        methodsComboBox.setModel(comboBoxModel);
    } catch (Exception e) {
        Splash.logger.warning(e.getMessage());
    }
}

private void methodComboEnable(){
    if(methodsComboBox.getSelectedItem().equals("CASH")){
        paidAmountInput.setEnabled(true);
    }
}

```

```

    }else{
        paidAmountInput.setEnabled(false);
        printInvoice.setEnabled(true);
    }
}
private void updateSummary(){
    int rowCount = invoiceTable.getRowCount();
    itemsLabel.setText(String.valueOf(rowCount));
    double totalAmount = 0;
    for(int i=0; i < rowCount; i++){
        totalAmount = totalAmount + (Double)invoiceTable.getValueAt(i, 4);
    }
    totalAmountLabel.setText(String.format("%.2f", totalAmount));
}

```

4. Loggers

```

public Splash() {
    initComponents();
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
    progress();
    try {
        logger = Logger.getLogger(Splash.class.getName());
        handler = new FileHandler(String.valueOf(sdf.format(new Date()))+"log.1.log");
        //handler.setFormatter(new SimpleFormatter());
        logger.addHandler(handler);
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}

```

5. Product Report

Kuweni The Store						
No.22, Home Town Street, Malabe						
Product Report						
#	Name	Qty	Price	Color	Size	Status
1722697 532311_	fff	1000	5999.0	White	M	ACTIVE
1722697 611496_	fff	500	5999.0	White	S	INACTIVE
1722697 623943_	fff	500	5999.0	White	L	INACTIVE
1723577 900008_	Sakura white cap	10	4999.0	White	XXL	ACTIVE
1723625 659306_	ZD8 black mug	10	3999.0	Black	M	ACTIVE
1723626 816961_	hh	10	533.22	White	M	ACTIVE
1723627 394325_	gg	10	5999.0	White	M	ACTIVE
1723627 513327_	gg	4	5999.55	White	XL	ACTIVE

6. Invoice Report

Kuweni The Store

No.22, Home Town Street, Malabe

Invoice No : 1723955534700

Customer Name : Anuradhi Fernando

Date : 2024-08-18

Customer Mobile : 0763026654

Time : 10:04:24

Barcode	Product Name	Selling Price	Quantity	Price (Rs)
1723577900008_White_XXL	go - (White_XXL)	4999.0	2	9998.0
1723625659306_Black_M	fff - (Black_M)	3999.0	1	3999.0

Payment method : CASH

Items : 2

Total price : 13997.00

Paid amount : 15000.00

Balance : 1003.00

Thankyou !, come back again to the univers...

7. USE CASE DIAGRAM

